

Deep Learning for Autonomous Vehicles

Lecture 1: Introduction

A. Alahi



Life on earth depends on the freedom to move

But... is it Safe? Sustainable? Accessible to all?



Deep Learning

Welcome in DLAV

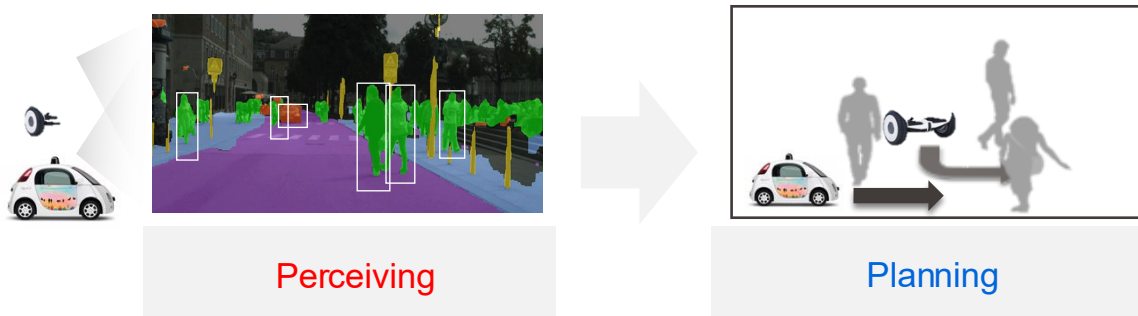
- Who?
- Why?
- What is it about?
- Administrative



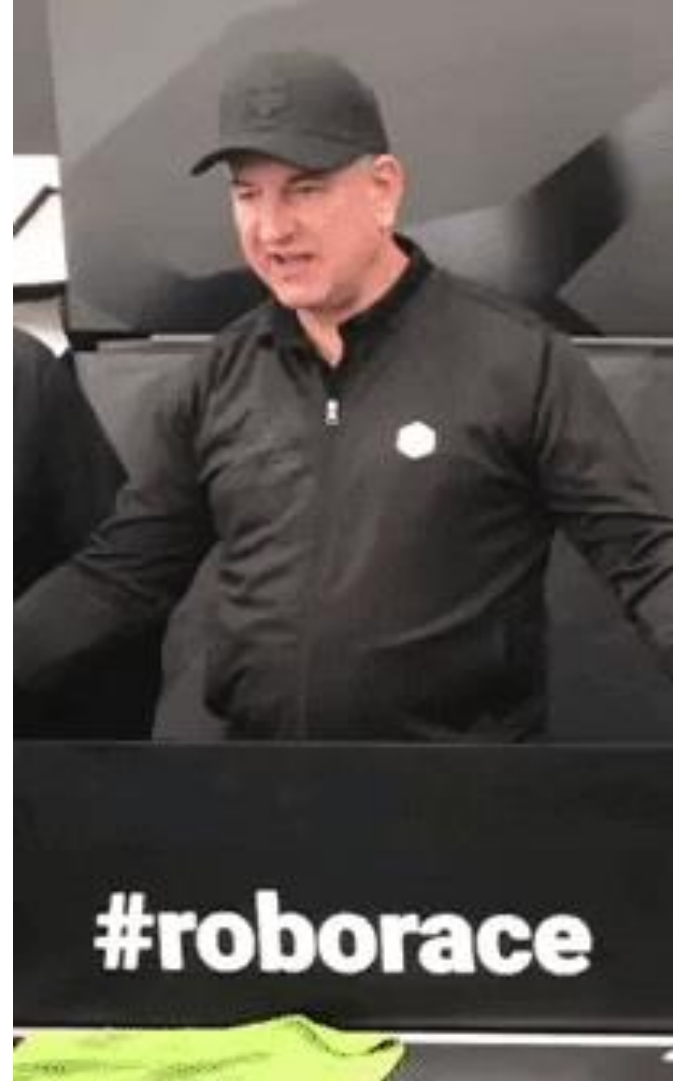
Ph.D. students

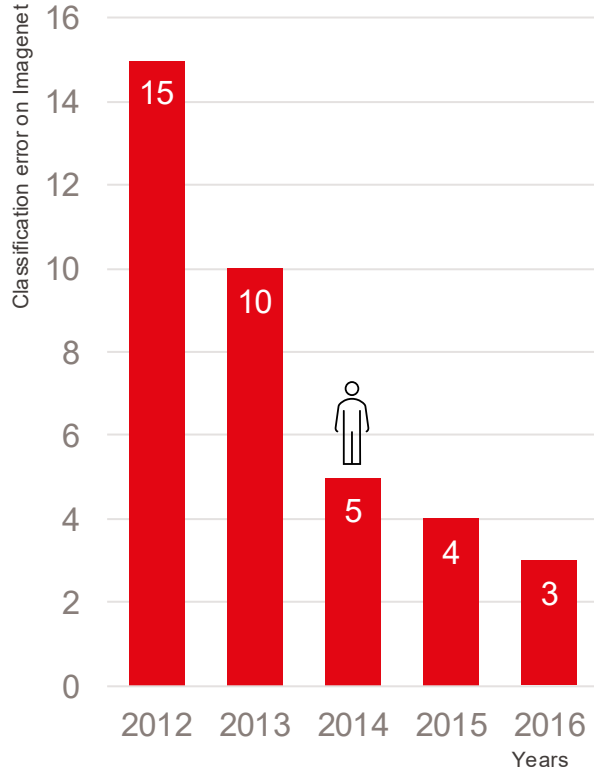
Po-Chien LuanMohamed Ossama Ahmed AbdelfattahLan FengMariam HassanMegh ShuklaYasamin Borhani

“AI = any device that perceives its environment and takes actions for its goals” [1]



How good is the current AI of AV?





AARJAN MARSHALL BUSINESS DEC 7, 2020 4:00 PM

Uber Gives Up on the Self-Driving Dream

The ride-hail giant invested more than \$1 billion in autonomous vehicles. Now it's selling the unit

The New York Times

Lyft Sells Self-Driving Project, Cutting a Big Expense

Published April 26, 2021 Updated Oct. 8, 2021

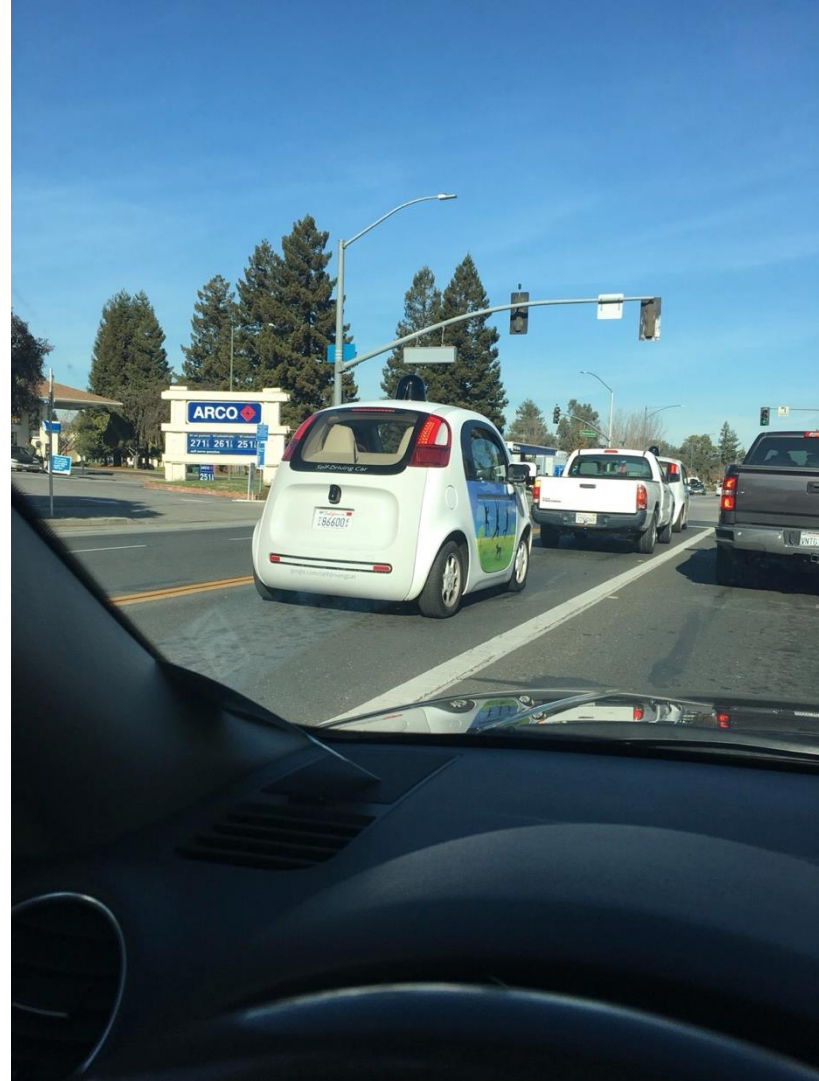
Two major U.S. companies are cutting back on their robotic last-mile delivery efforts.

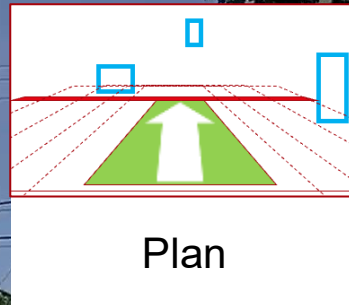
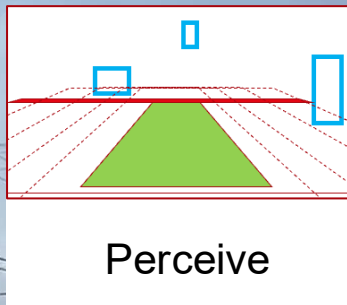
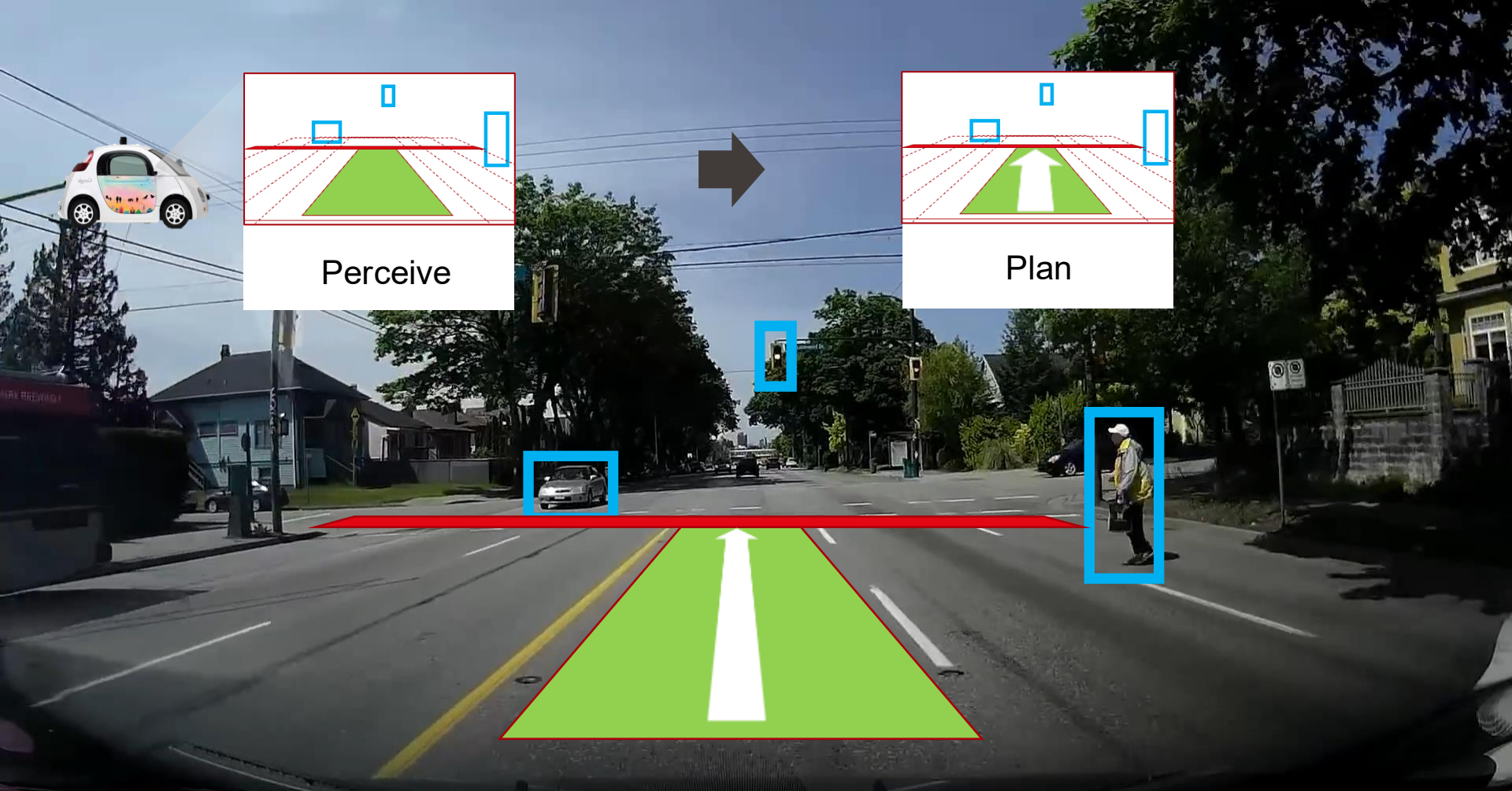
By [Cesareo Contreras](#) · 🕒 October 7, 2022

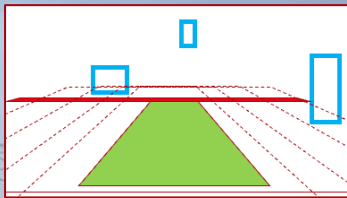
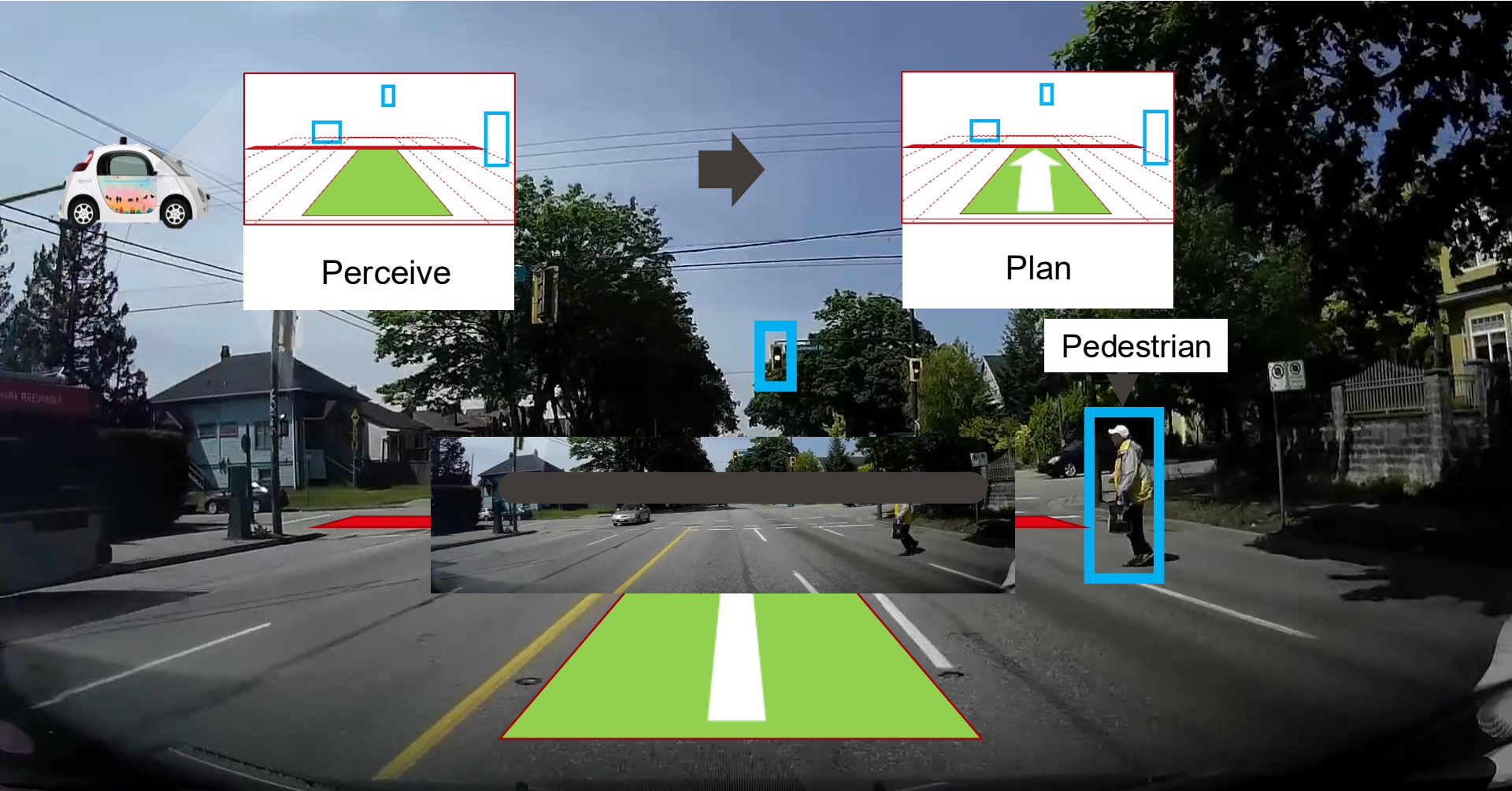


Oct. 7th 2022

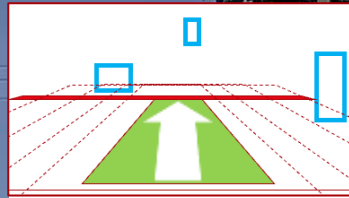
We have self-driving cars in Mountain View...







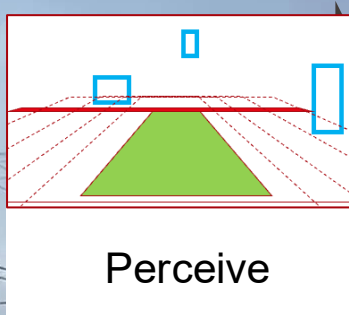
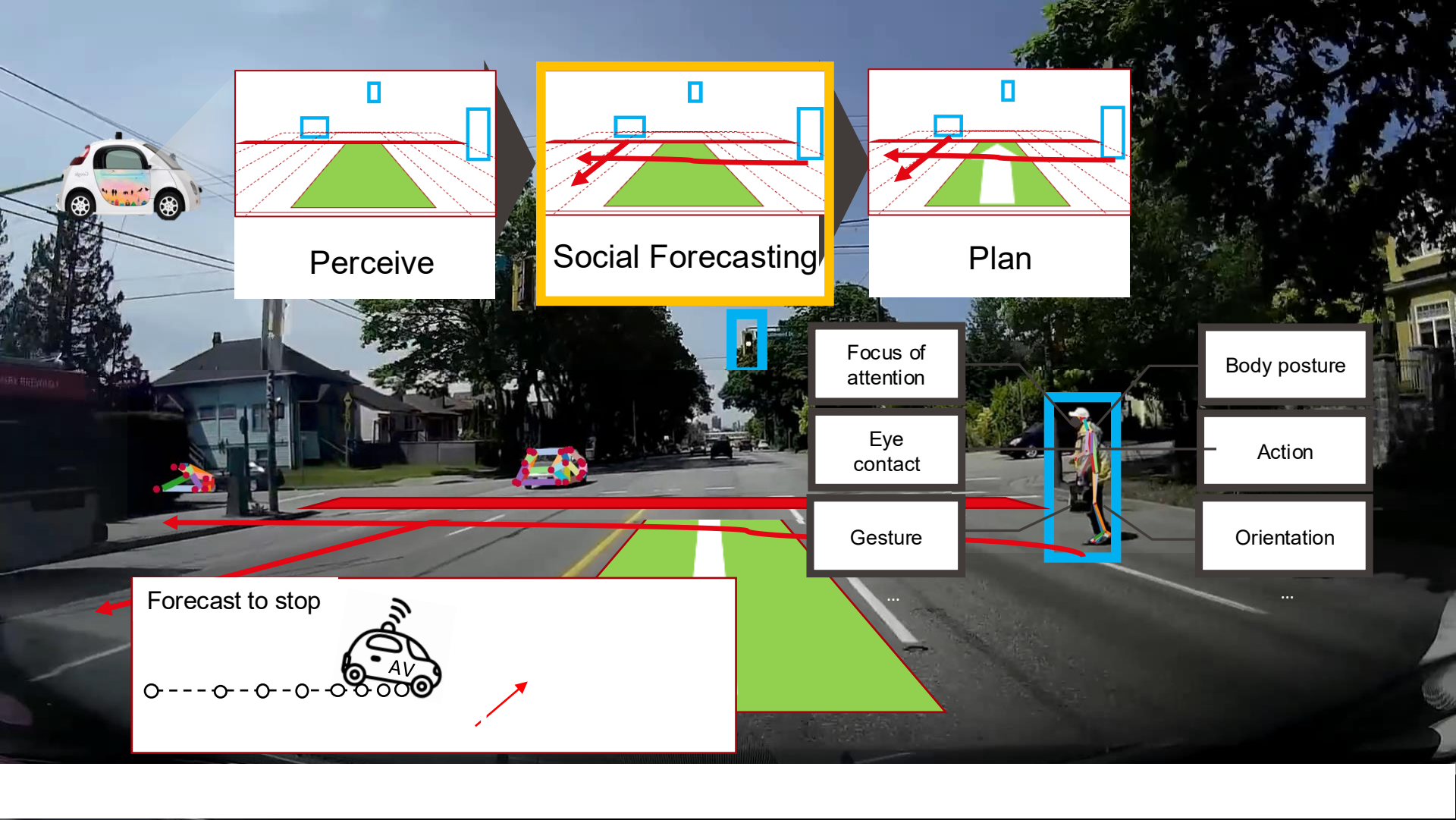
Perceive



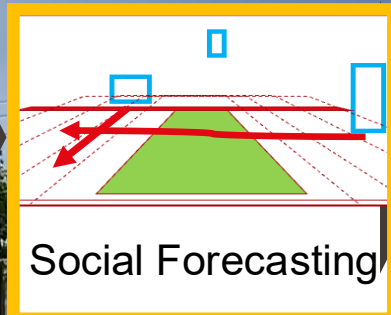
Plan

Pedestrian

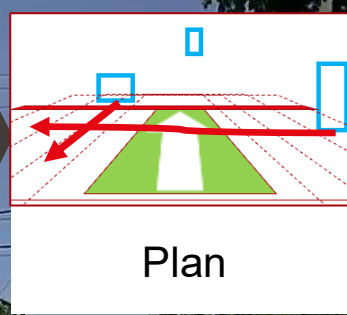




Perceive



Social Forecasting



Plan

Focus of
attention

Eye
contact

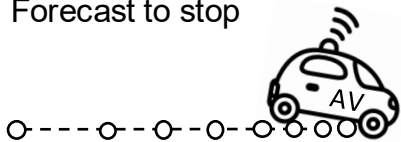
Gesture

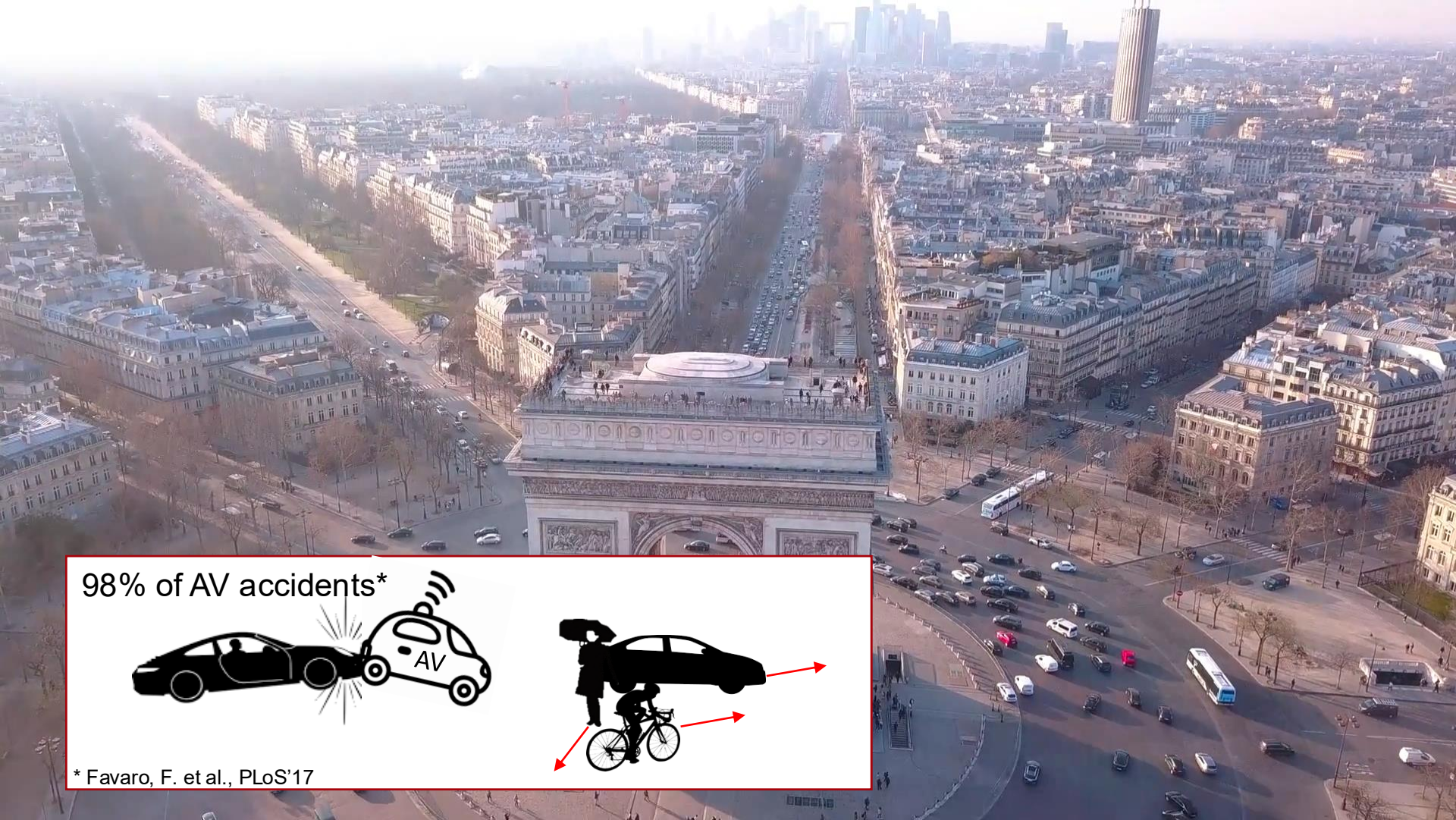
Body posture

Action

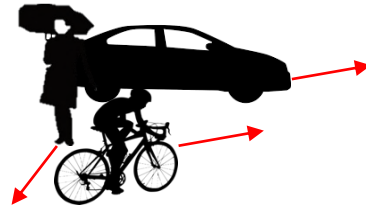
Orientation

Forecast to stop





98% of AV accidents*



* Favaro, F. et al., PLoS'17



Intelligent Agents needs to understand social interactions



“Man is by nature a social animal” - Aristotle



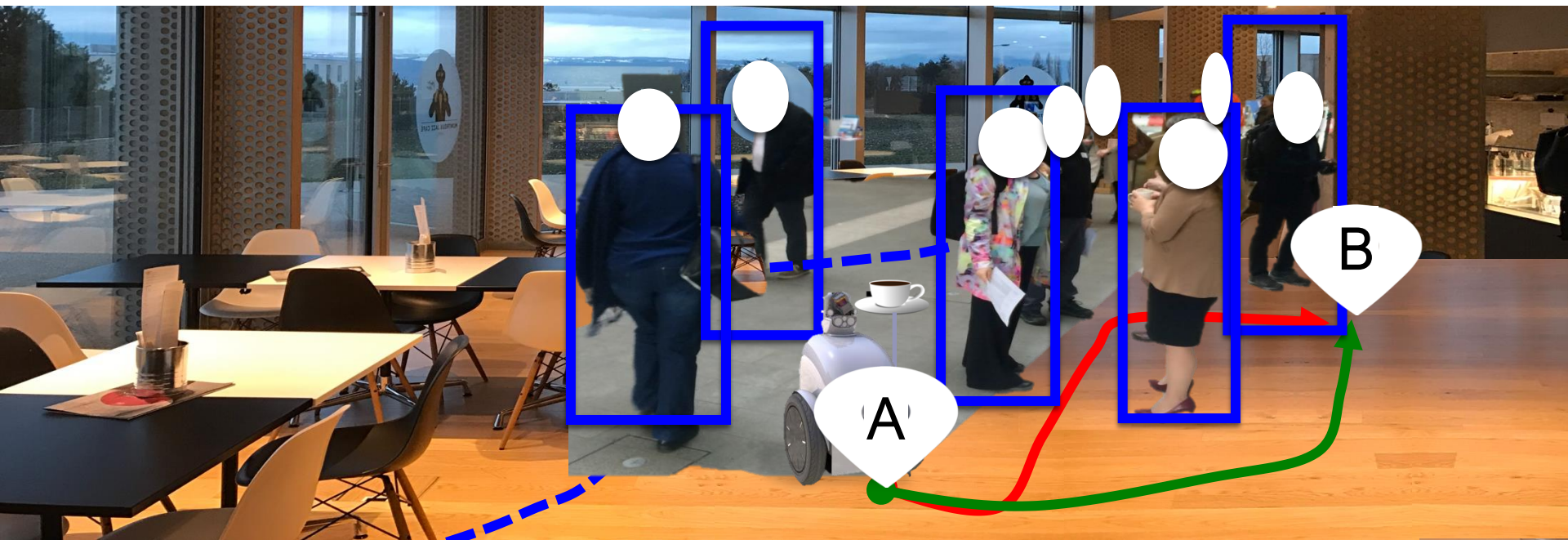
How can **machines** make the same decisions as **humans** do in **social** scenes?



How can **machines** predict human social interactions?



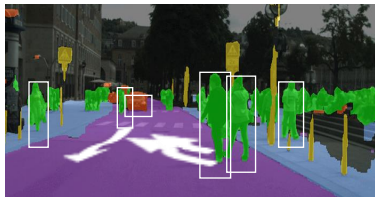
Socially-aware AI = AI + Social Intelligence



“... is the capacity to **effectively** navigate and negotiate complex social **relationships** and environments” [1]



What is the course about?

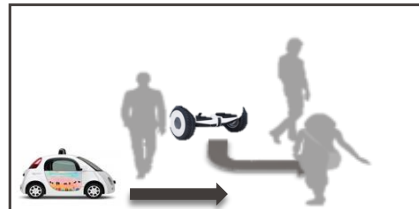


Prediction



Perceiving

**Socially-aware
AI**



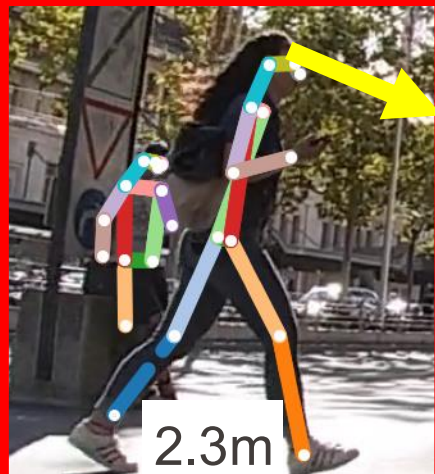
Planning

Object detection



Body poses + Activities + Relationships

Walking



Standing



Standing



Talking

Walking

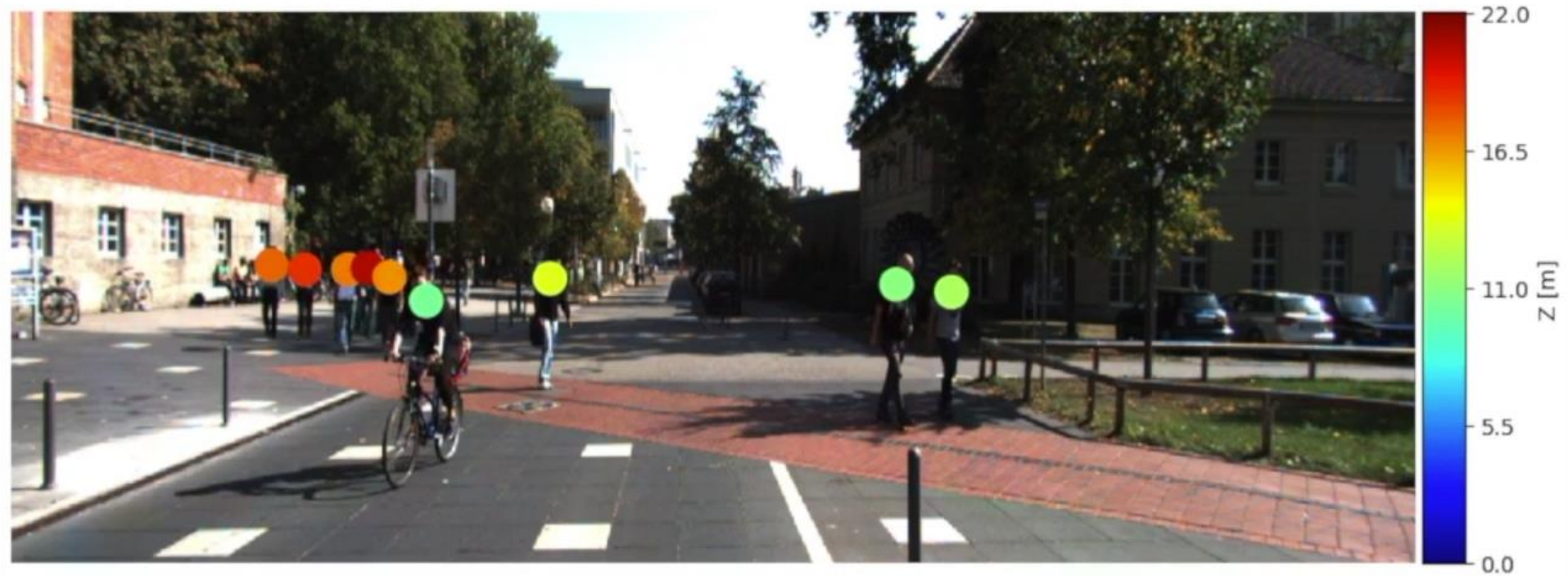


[1] PifPaf: Composite Fields for Human Pose Estimation, **CVPR'19**; on-line demo: vitademo.epfl.ch/movements

[2] Keypoints communities, **ICCV'21**; [3] OpenPifPaf, open-source library, **IEEE ITS'21**

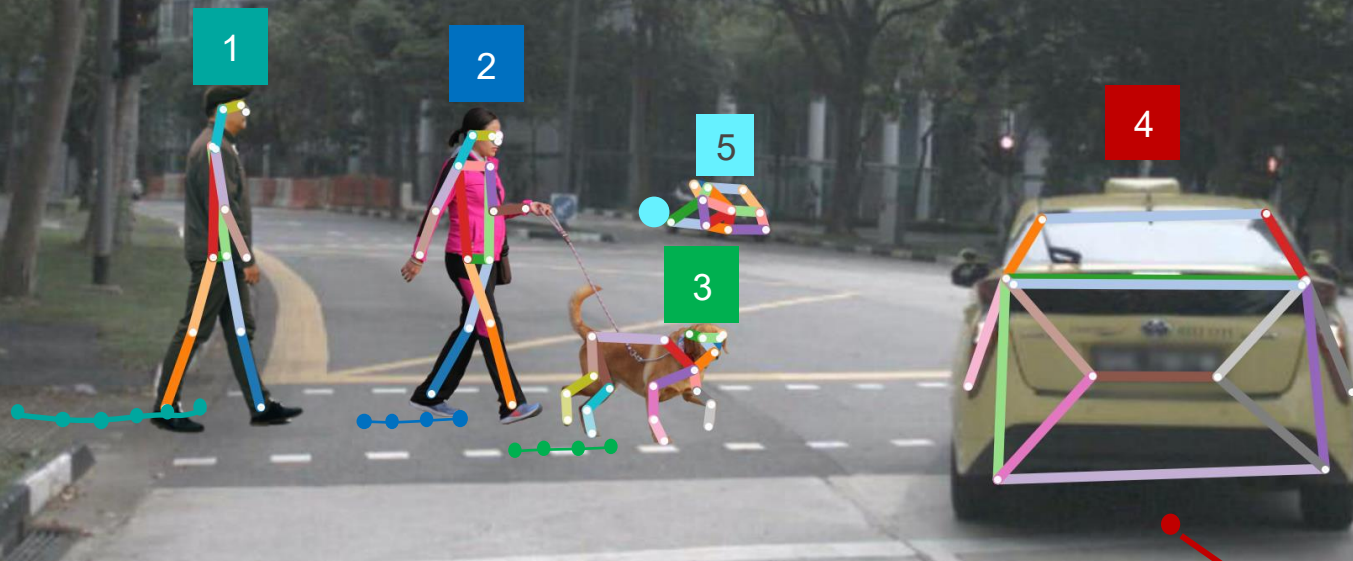
[4] Convolutional Relational Machine, **CVPR'19**; [5] Detecting 32 Pedestrian Attributes, **IEEE ITS'21**

[6] Monoloco: Monocular 3D pedestrian localization and uncertainty estimation, **ICCV'19**; [7] MonStereo, **ICRA'21**



[1] Monoloco: Monocular 3D pedestrian localization and uncertainty estimation, **ICCV'19**; [2] MonStereo, **ICRA'21**

Pose detection with tracking




[← Back to all posts](#)

February 22, 2022

Utilizing key point and pose estimation for the task of autonomous driving

TECHNOLOGY

The Waymo Team



Imagine: you're driving a car and you see a person standing on the corner of a street. How do you know if they're going to cross? Interpreting another road user's intent and actions can be challenging and complex, even for human drivers. It is a driver's job to gauge whether another road user wants you to wait and let them cross, whether they are waiting for you to cross after you pass, or if they are just waiting there for a different reason. Even then, what an individual signals might differ from the action they complete. To help navigate these nuanced situations, one of the important signals the Waymo Driver uses is key points.

Making more accurate and efficient models with less compute

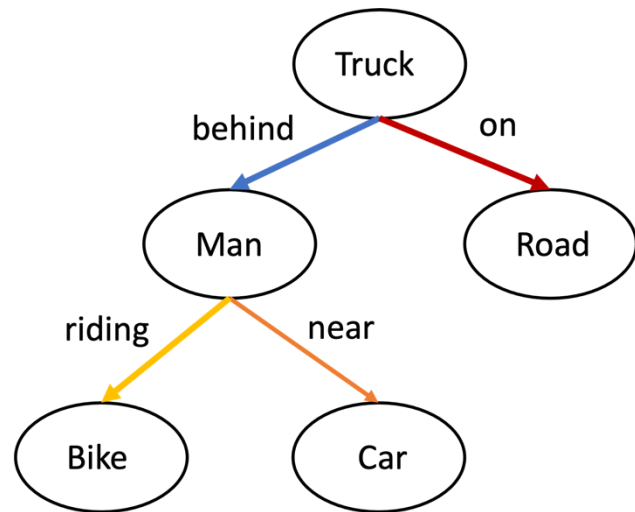
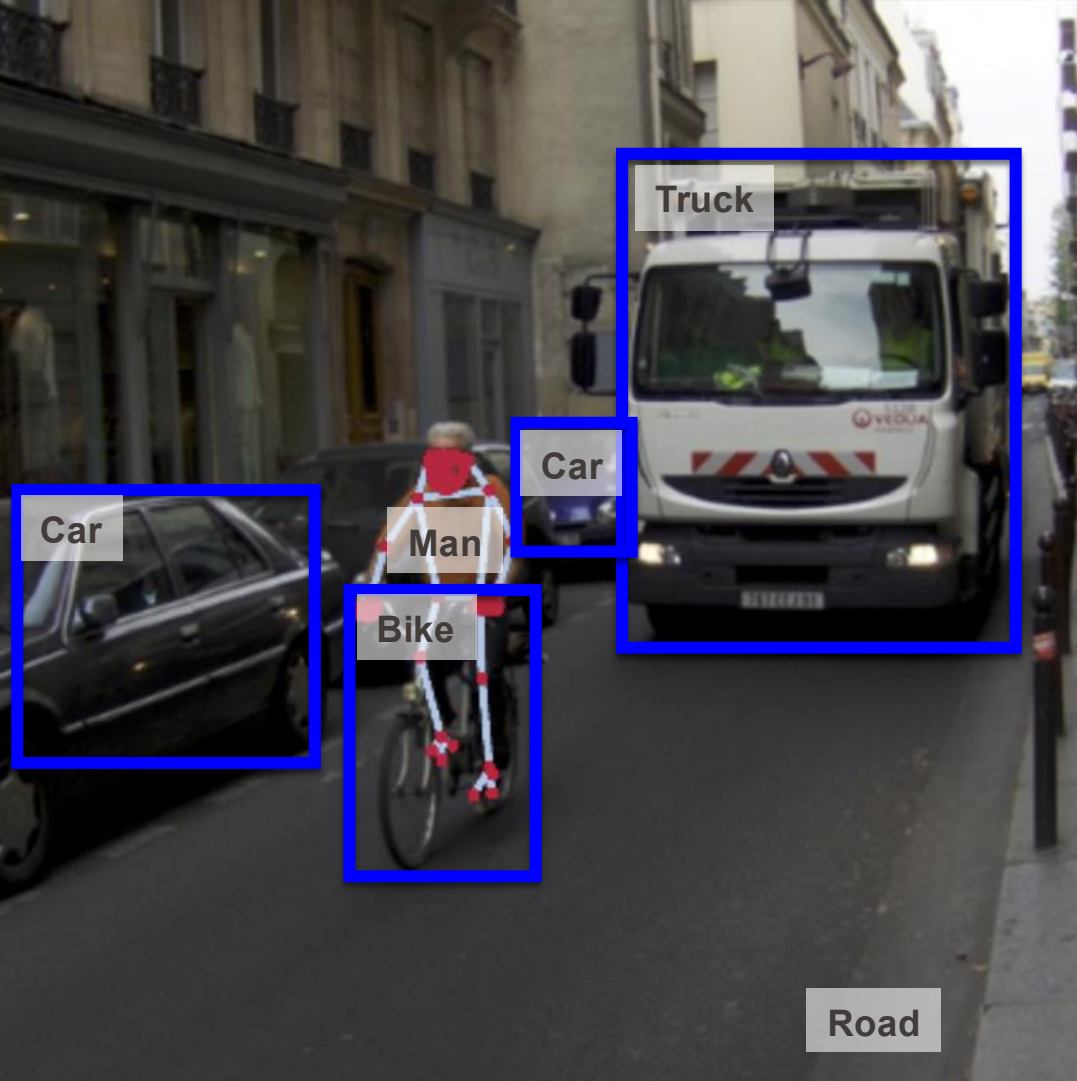
Key points are a simplified way to represent the complex human form using a limited number of points across the body.







Scene Graph Generation



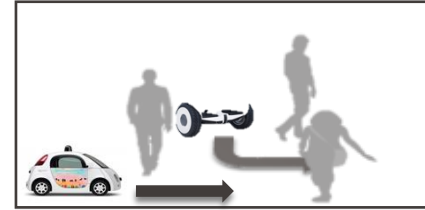


Social Forecasting



Perceiving

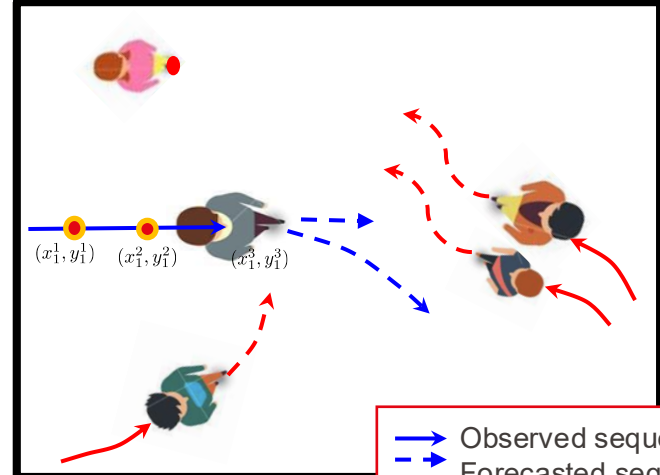
**Socially-aware
AI**



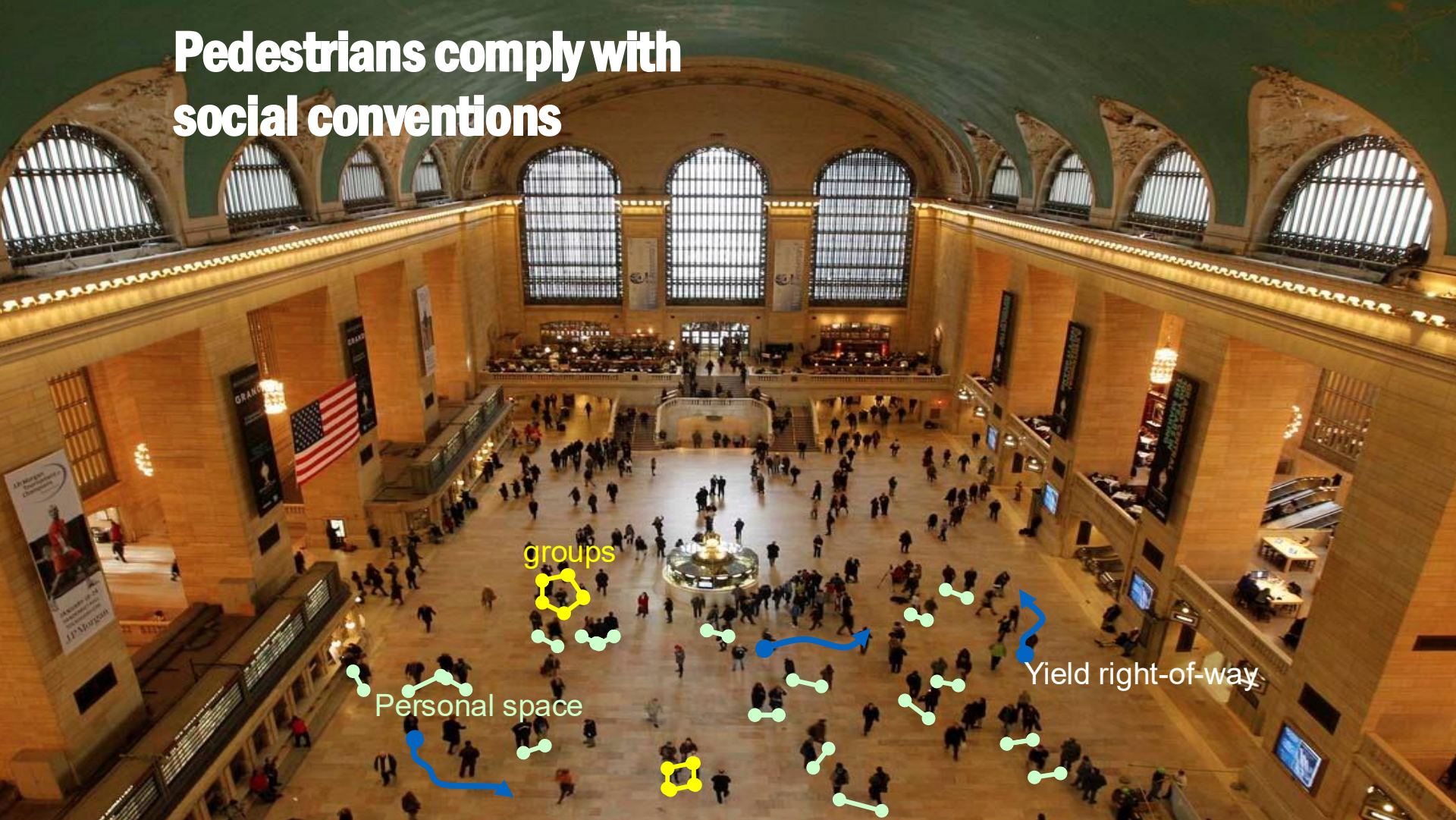
Planning

Social Forecasting (w/ pedestrians)

- **Input:** several sequences of states
- **Output:** forecast the future states, e.g., next 5 seconds



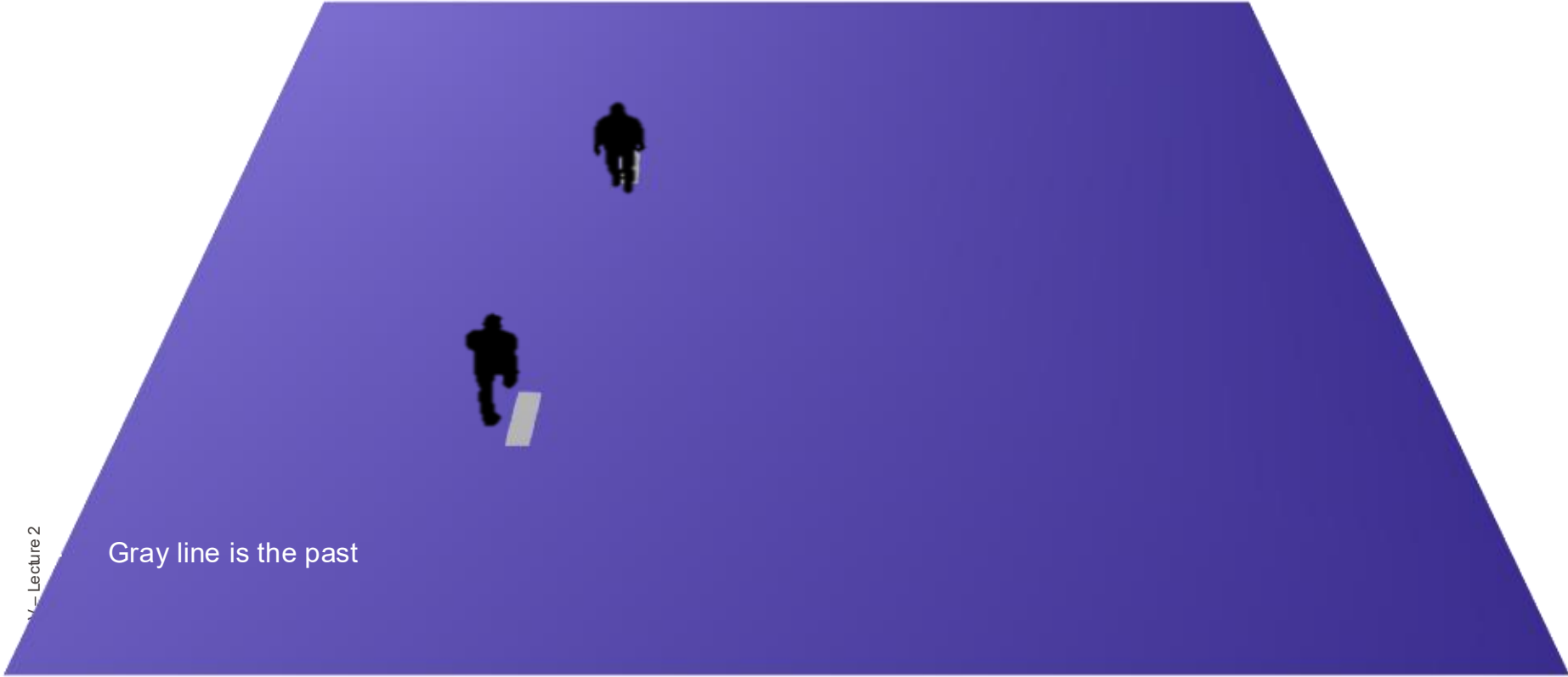
Pedestrians comply with social conventions

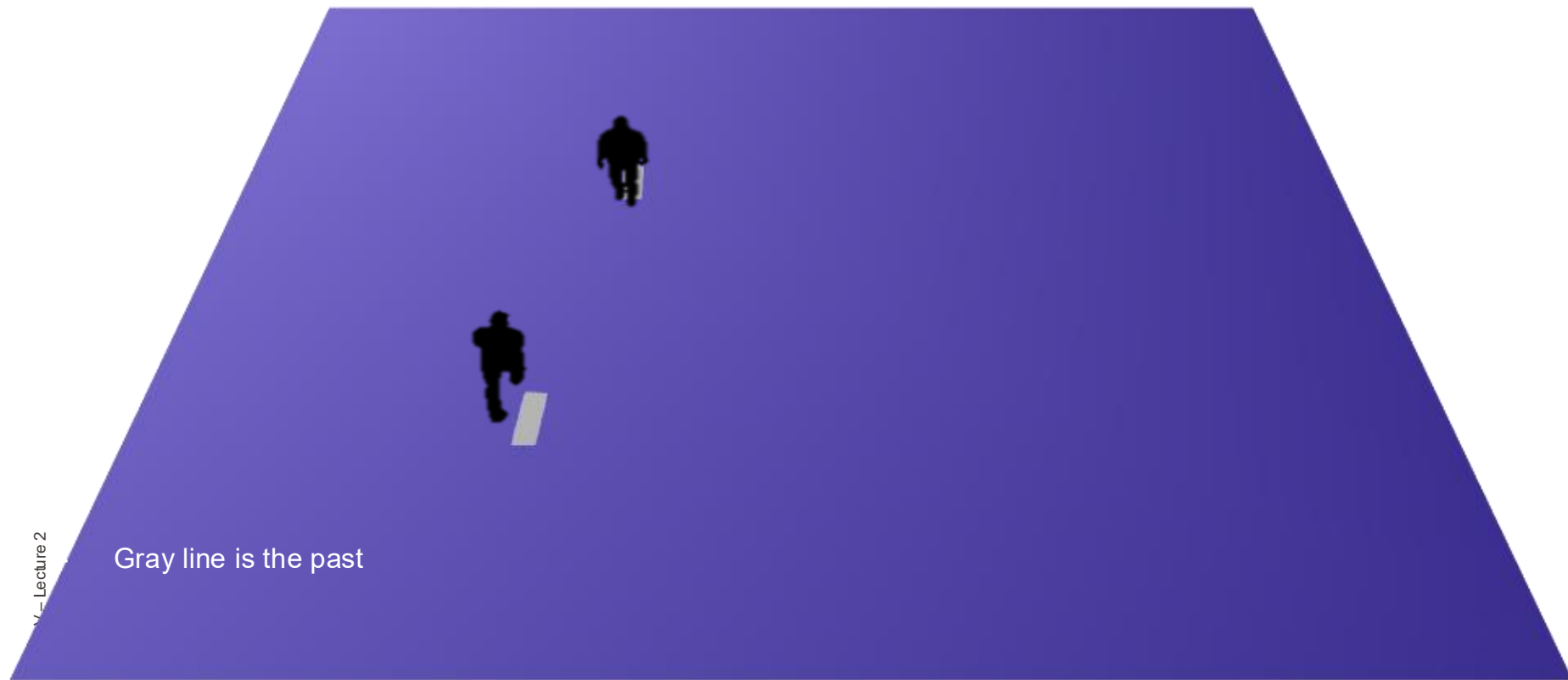


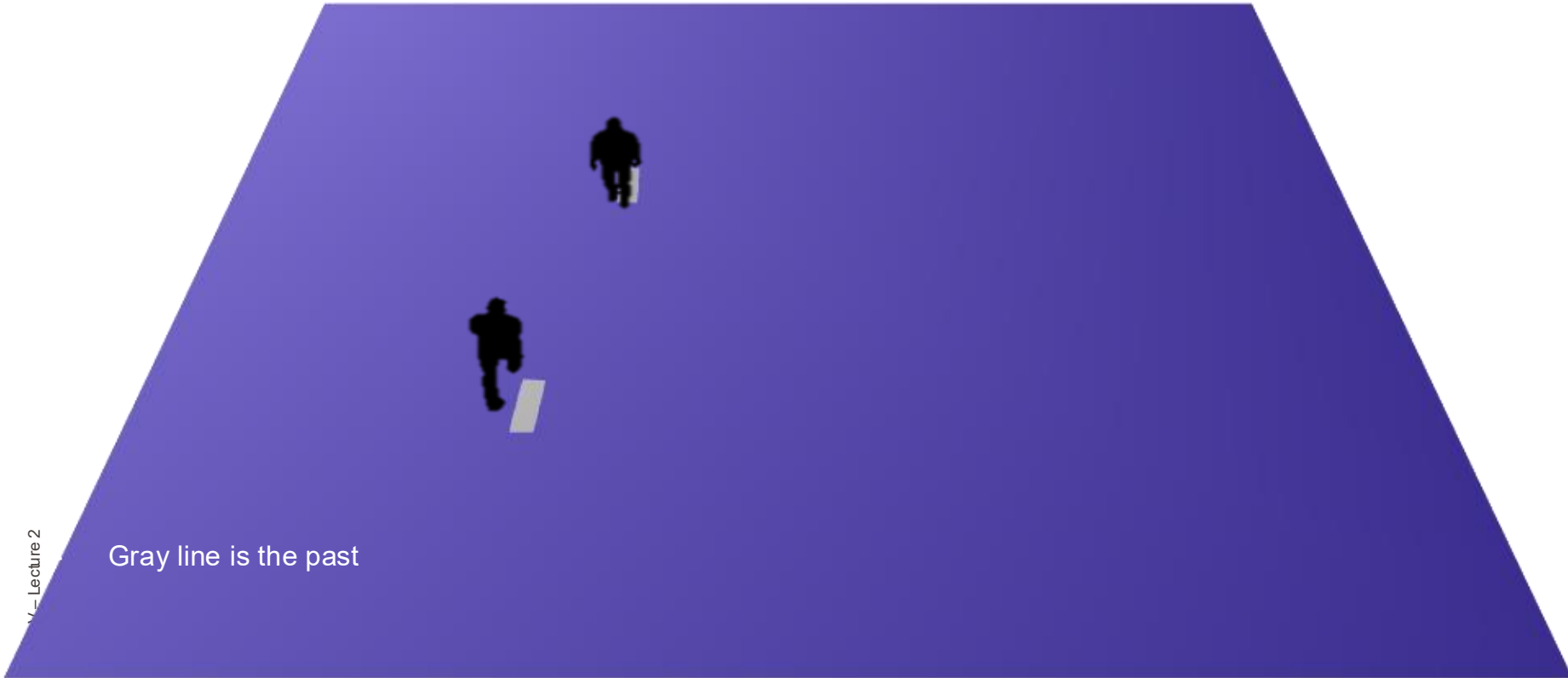
Two pedestrians



Gray line is the past









Can you predict the future?



Option 1

Gray line is the past

Can you predict the future?



Option 2

Gray line is the past
Black line is the future

Can you predict the future?



Option 3

Gray line is the past
Black line is the future

The future is...



Option 3 !

Gray line is the past
Black line is the future



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past







- Black line is the ground truth trajectory
- Gray line is the past



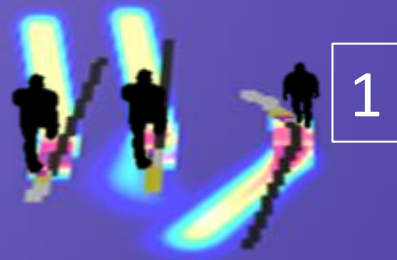
- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



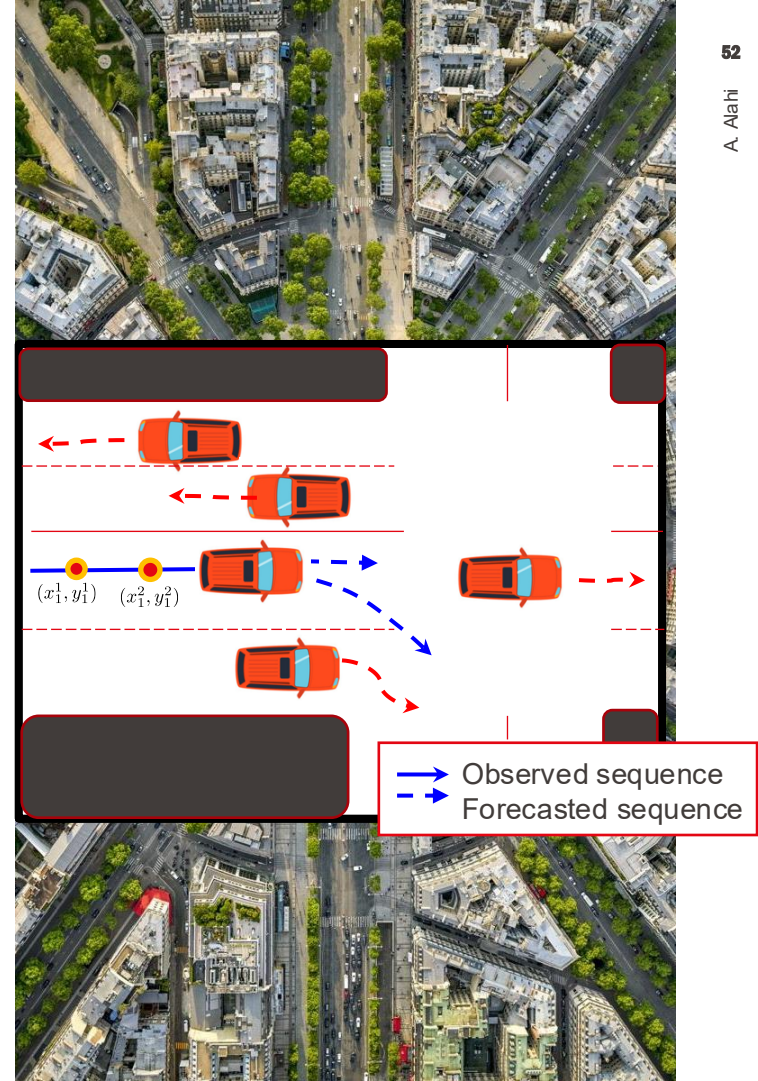
- Black line is the ground truth trajectory
- Gray line is the past

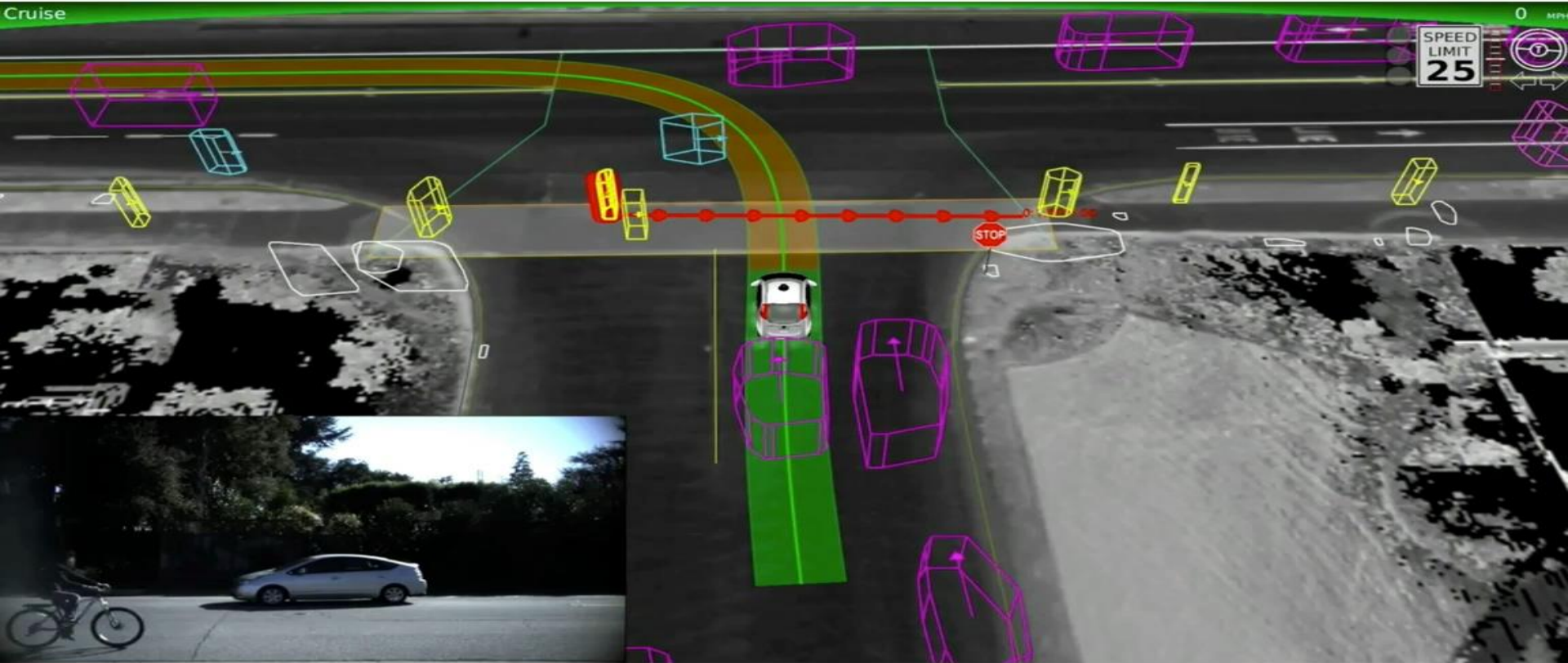


- Black line is the ground truth trajectory
- Gray line is the past

Social Forecasting (w/ vehicles)

- **Input:** several sequences of states
+ scene infrastructure
- **Output:** forecast the future states,
e.g., next 5 seconds





ground truth
linear
Vanilla MLP
MLP + Scene

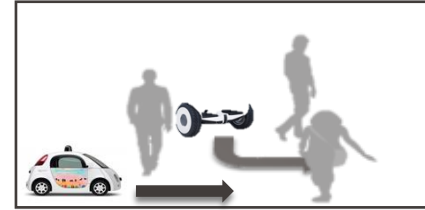
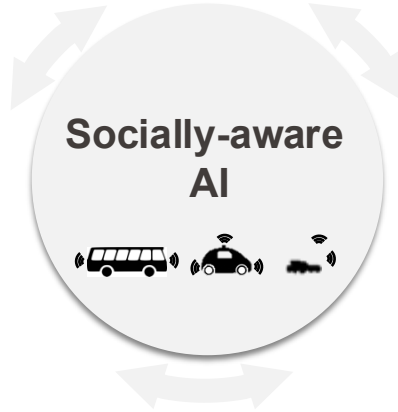




Perceiving

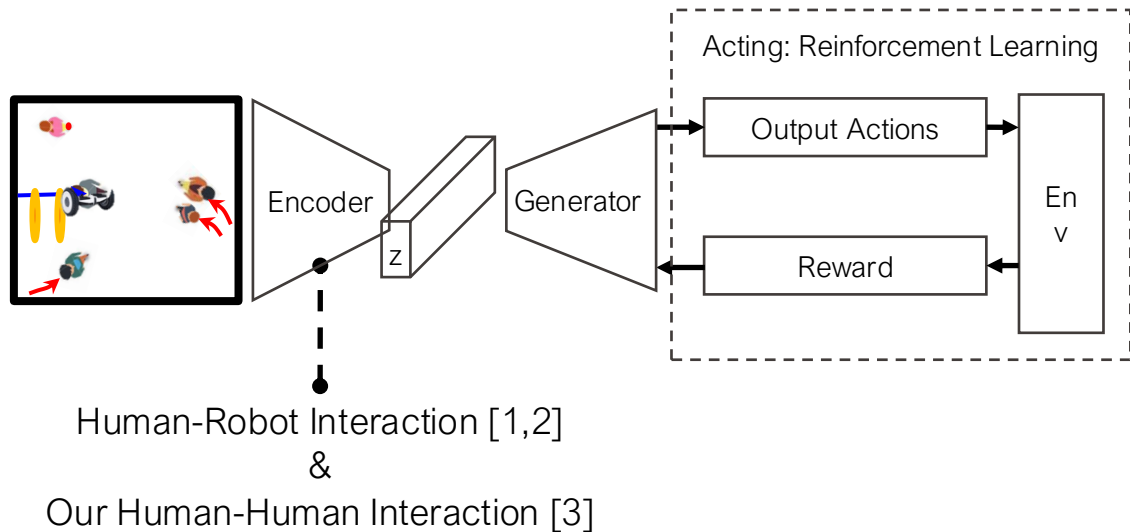


Social Forecasting



Planning

Crowd-Robot Interaction [1]



Previous works

[1] HRI, Chen, C., *et al.*,

IROS'17

[2] HRI, Everett, M., *et al.*,

IROS'18

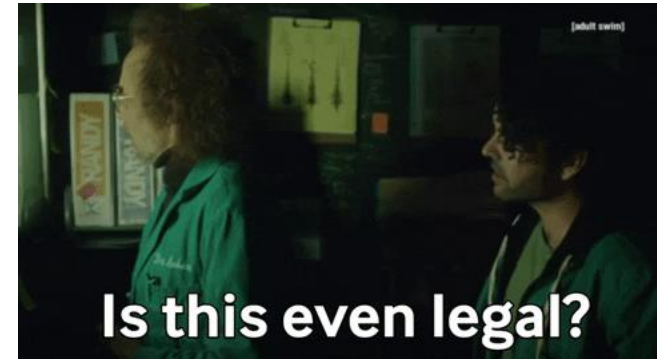
Our work

[3] Crowd-Robot Interaction,

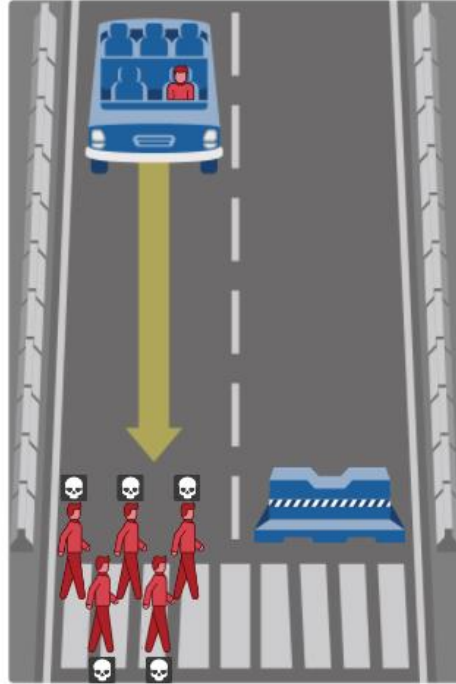
ICRA'19

With:

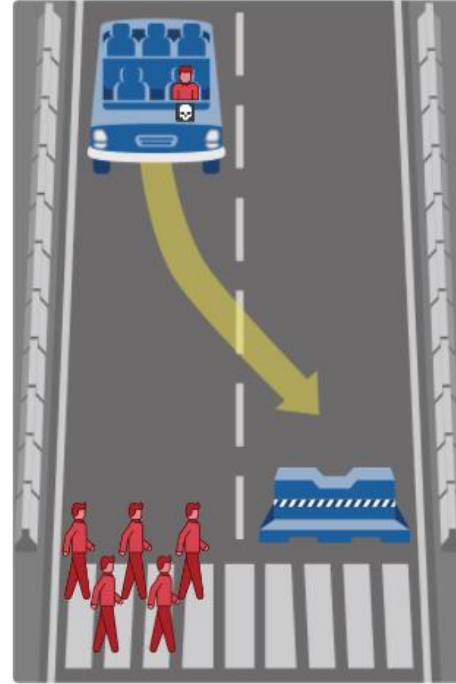
- **Social** awareness
- **Ethical** awareness



Q2: Option A or B?



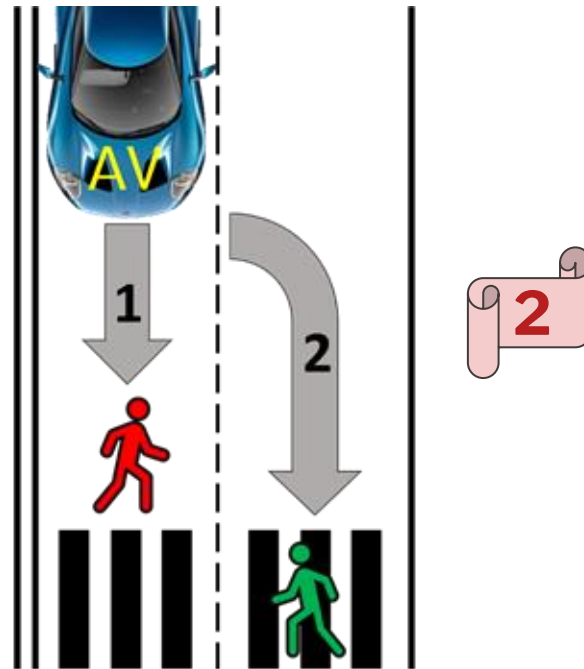
A



B

Do you prefer the AV to:

- ☐ 1. Save the person on the zebra crossing
- ☐ 2. Save the person crossing illegally



☐ Legality

☐ Relation to AV

☐ Amount of people

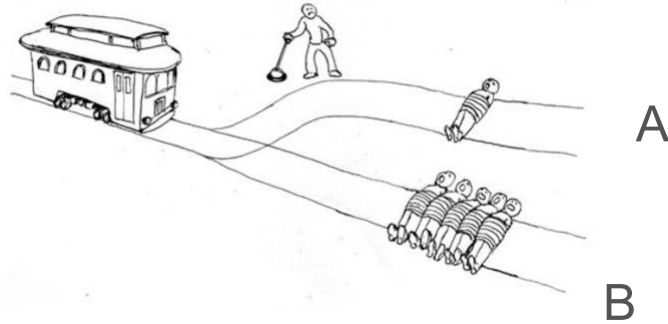
☐ Position from AV

☐ Probability of fatality

☐ Things involved in the collision

The trolley problem

Q1: Option A or B?



CIVIL-459

Admin

- Assessment methods:
 - Lab projects (Individual): 30%
 - Midterm (Individual): 30%
 - Final project (in group): 40%
 - Report
 - Competition
 - Presentation

- Pre-requisite:
 - Calculus, Linear Algebra. Probability and Statistics
 - Programing skills
 - **Python**

By the end of the course

HONDA
The Power of Dreams



TOYOTA
RESEARCH INSTITUTE



TESLA

DiDi

autoX

SAMSUNG

lyft

Uber Research

Valeo

CRUISE

pony.ai



WAYMO

ZOOX

CarPostal
La classe jaune.

WeRide

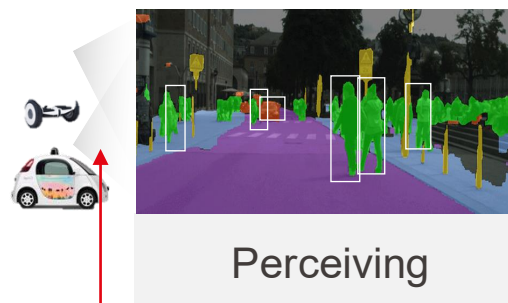
ARGO AI



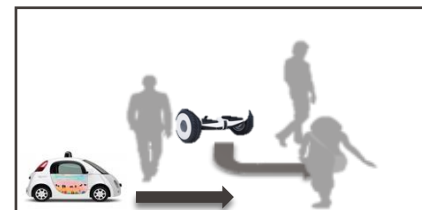
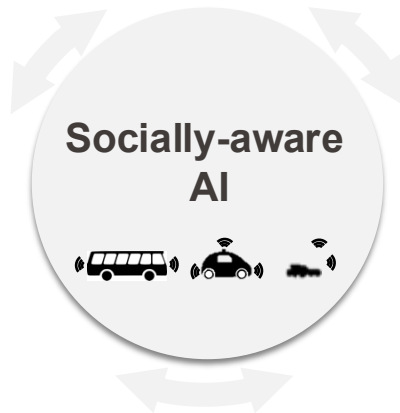
nuro

mobileye
An Intel Company





Social Forecasting



Planning

0-Sensing

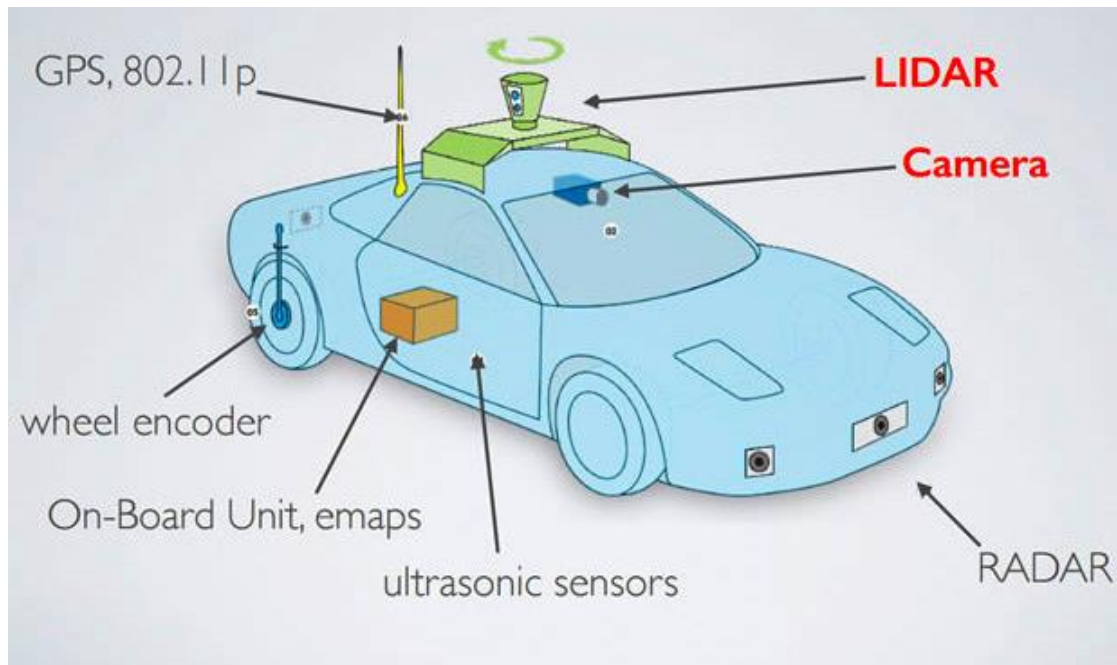
FROM A CAR



Acquiring/Collecting/Measuring
digital signal/data
about a
phenomenon/object/scene



FROM A CAR





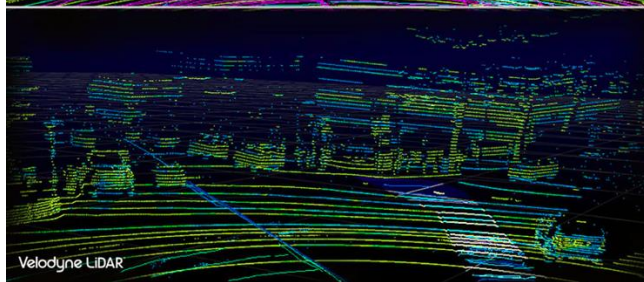
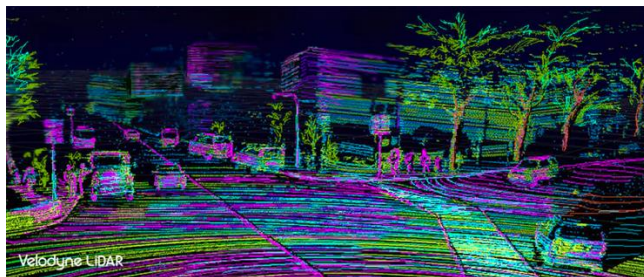
RGB



3D

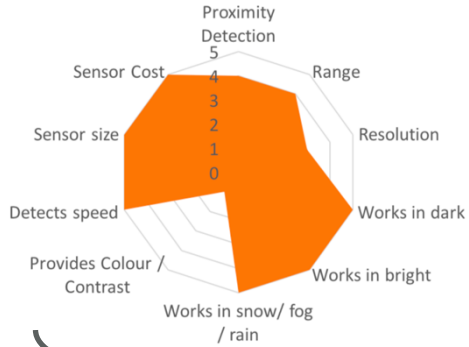


IR/Thermal

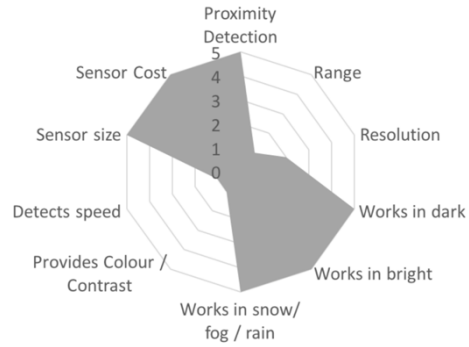




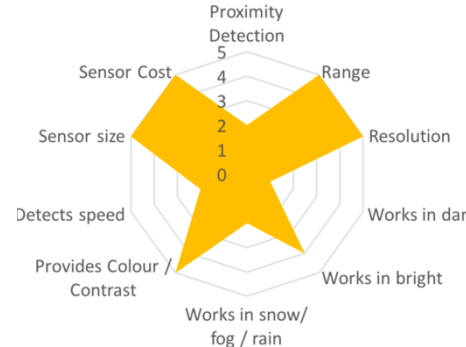
Radar



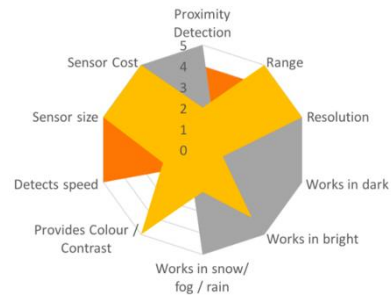
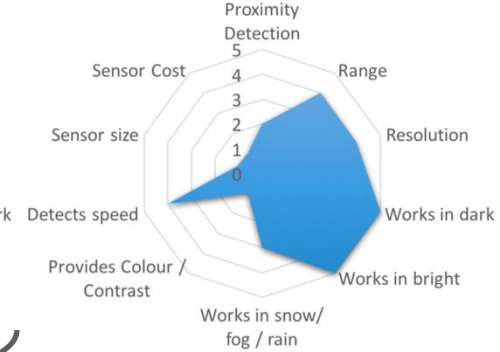
Ultrasonic



Passive Visual



Lidar



FROM A DRONE





HUMAN-SPACE INTERACTION



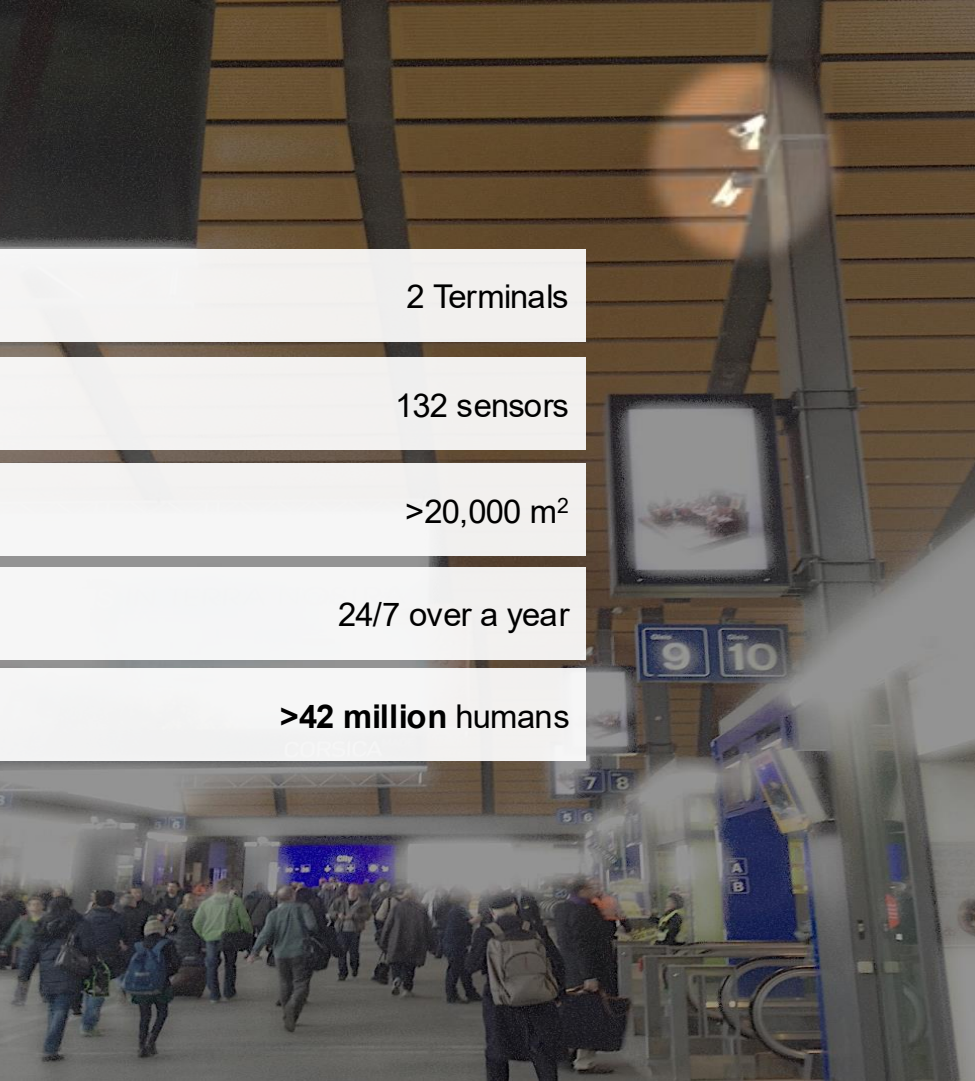
2 Terminals

132 sensors

>20,000 m²

24/7 over a year

>42 million humans

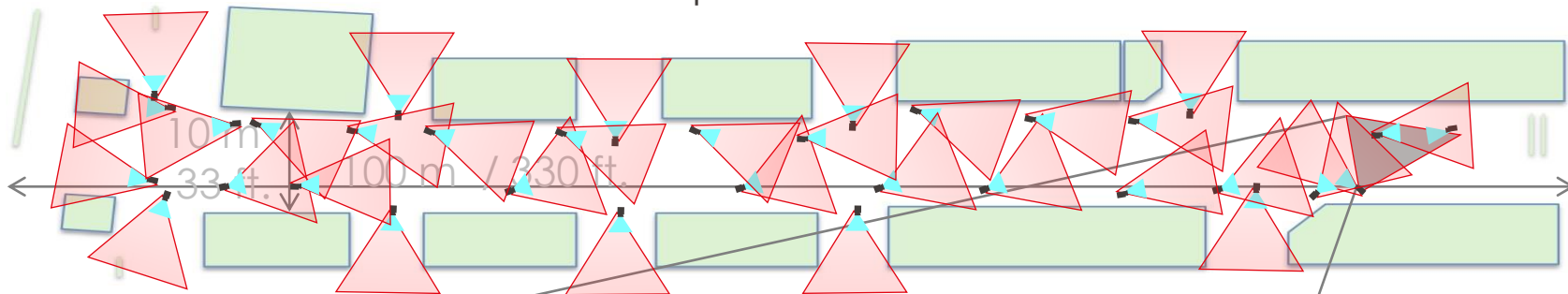


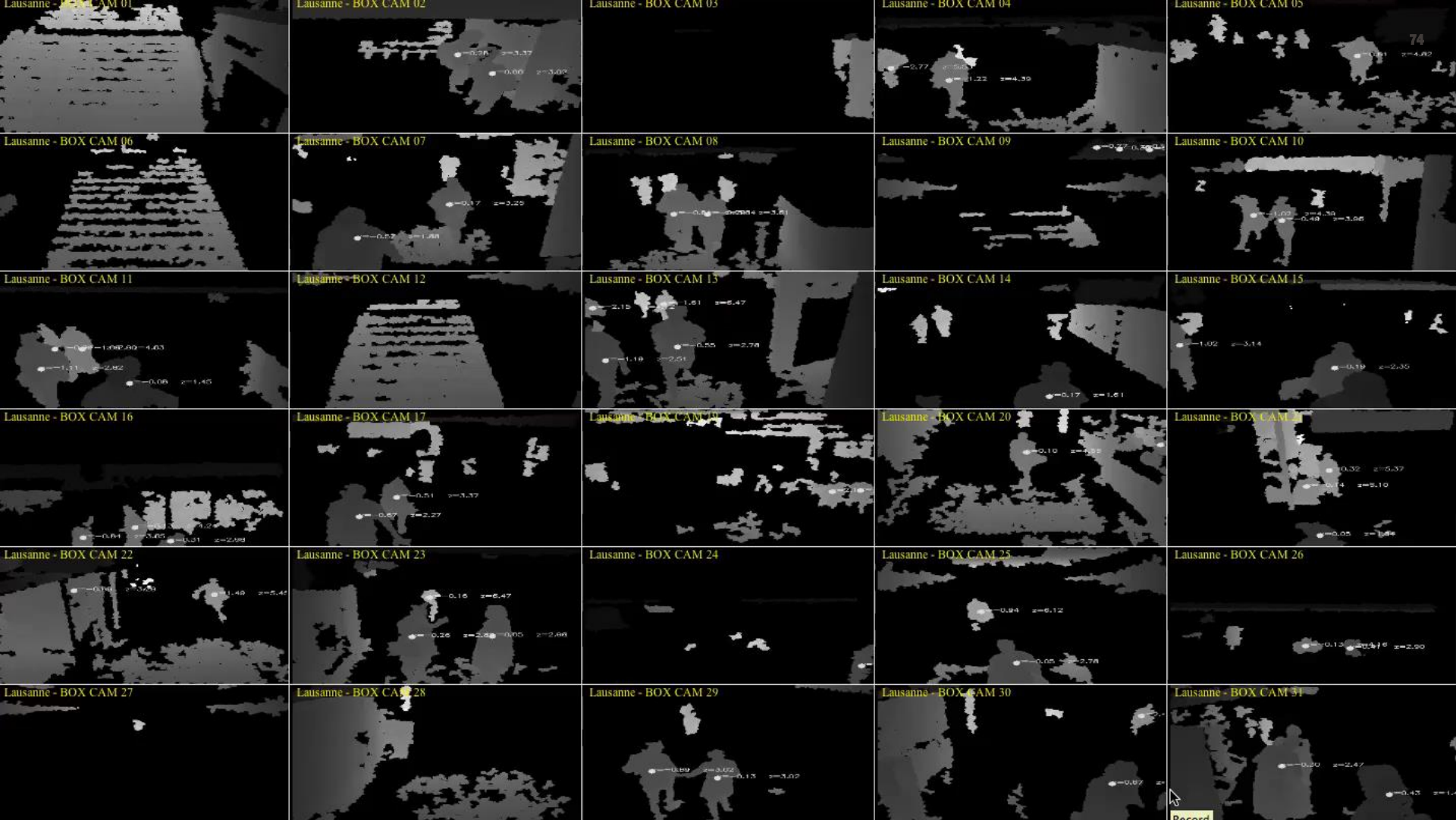
(Lausanne, Basel)

A corridor with 32 sensors



Top View





Basel - IP CAM 01



Basel - IP CAM 02



Basel - IP CAM 03



Basel - IP CAM 04



Basel - IP CAM 05



Basel - IP CAM 06



Basel - IP CAM 07



Basel - IP CAM 08



Basel - IP CAM 09



Basel - IP CAM 10



Basel - IP CAM 11



Basel - IP CAM 12



Basel - IP CAM 13



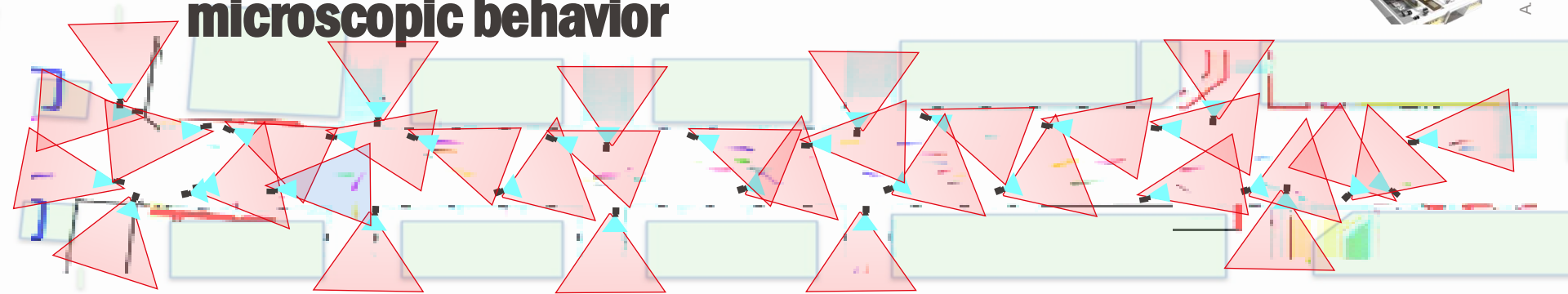
Basel - IP CAM 14



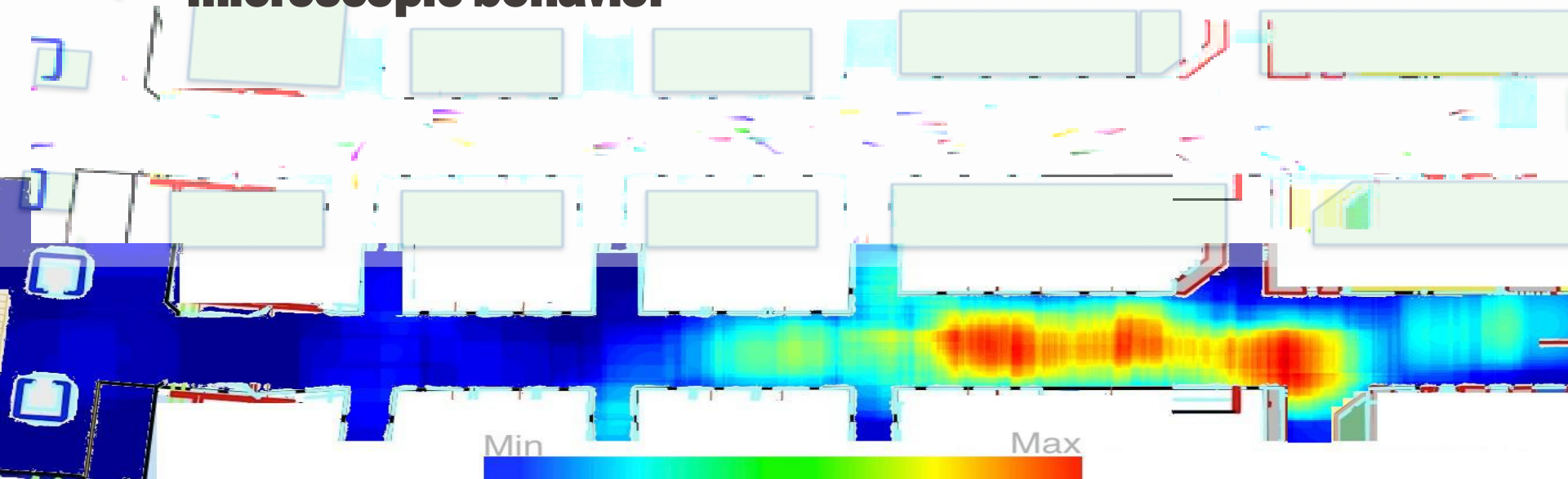
Basel - IP CAM 15



Sensing commuters' microscopic behavior



Sensing commuters' microscopic behavior



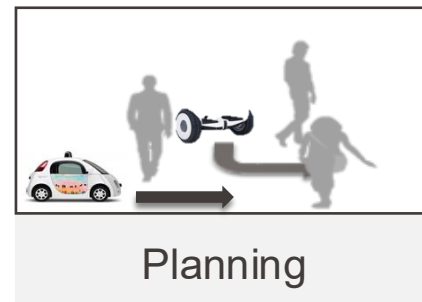
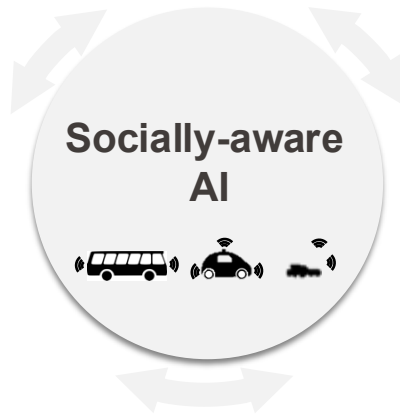
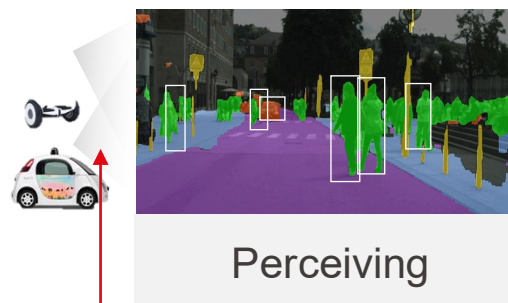
People
42 million

Av. duration
1 minute

Av distance
100m

Density (up to)
1 pedestrian/m²

Paths (O/D)
196



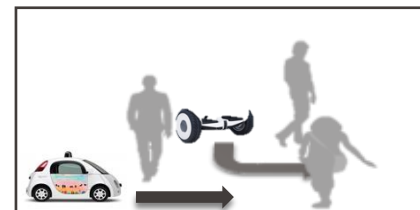
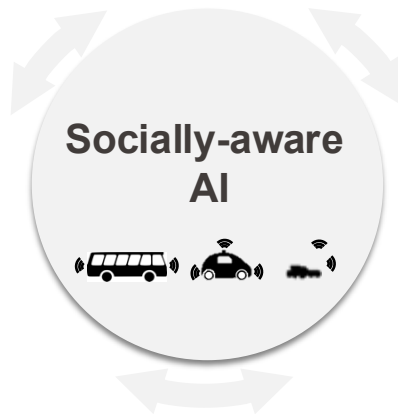
0-Sensing



Social Forecasting



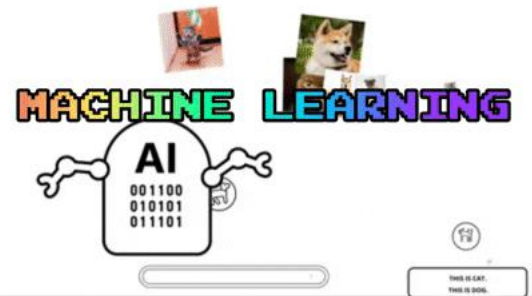
Perceiving



Planning

- Image classification
- Quick recap of Machine Learning (ML)
 - Nearest Neighbor
 - Linear classifier
 - Loss functions
- Optimization methods (find the best W)

GIGABYTE™



Given a set of discrete labels (aka choice set): pedestrian, car, truck, dog...

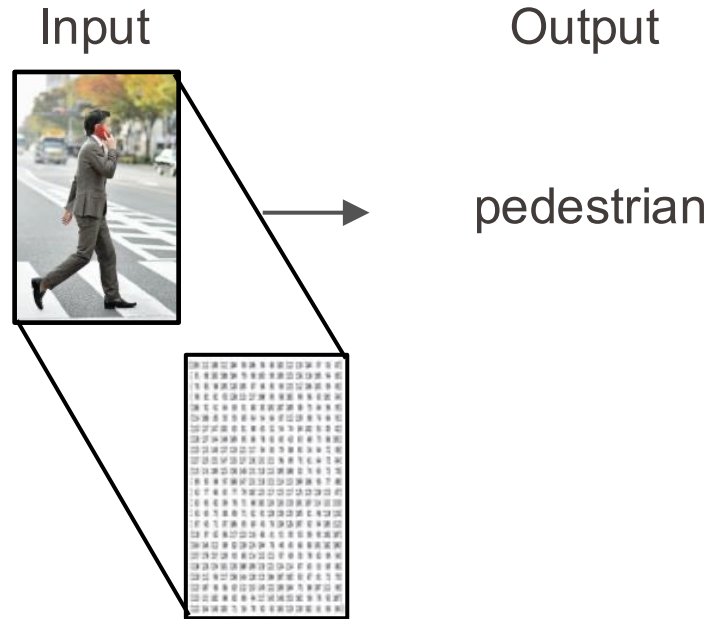


Image Classification Challenges

Given a set of discrete labels (aka choice set): pedestrian, car, truck, dog...

Input



Output

pedestrian

Change in:
• Viewpoint



Image Classification Challenges

Given a set of discrete labels (aka choice set): pedestrian, car, truck, dog...

Input



Output

pedestrian

Change in:

- Viewpoint
- Illumination



Image Classification Challenges

Given a set of discrete labels (aka choice set): pedestrian, car, truck, dog...

Input



Output

pedestrian

Change in:

- Viewpoint changes
- Illumination
- Deformation



Image Classification Challenges

Given a set of discrete labels (aka choice set): pedestrian, car, truck, dog...

Input



Output

pedestrian

Change in:

- Viewpoint changes
- Illumination
- Deformation
- Occlusion



Image Classification Challenges

Given a set of discrete labels (aka choice set): pedestrian, car, truck, dog...

Input



Output

pedestrian

Change in:

- Viewpoint changes
- Illumination
- Deformation
- Occlusion
- Intraclass variation
- ...

Noises

aka

Variations

aka

Deformations

aka

Nuisances



Image Classification Challenges

Given a set of discrete labels (aka choice set): pedestrian, car, truck, dog...

Input

Output



pedestrian

Change in:

- Viewpoint changes
- Illumination
- Deformation
- Occlusion
- Intraclass variation
- ...

Noises

aka

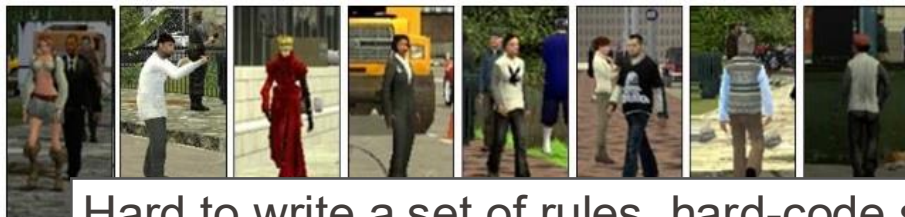
Variations

aka

Deformations

aka

Nuisances



Hard to write a set of rules, hard-code steps for recognizing pedestrians

Image Classification

Some Attempts

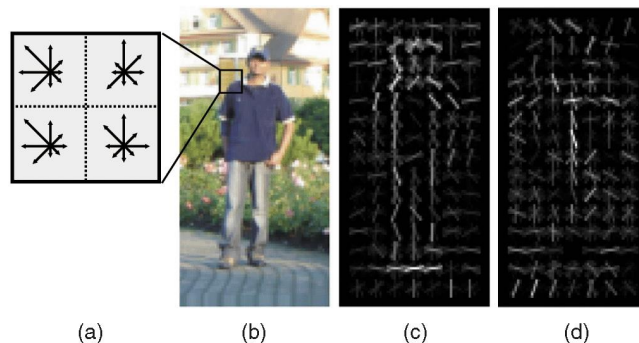
Given a set of discrete labels (aka choice set): pedestrian, car, truck, dog...

Input



Output

pedestrian



Hard to write a set of rules, hard-code steps for recognizing pedestrians

Machine learning to the rescue

- Tom Mitchell definition (1998) of a well posed learning problem:

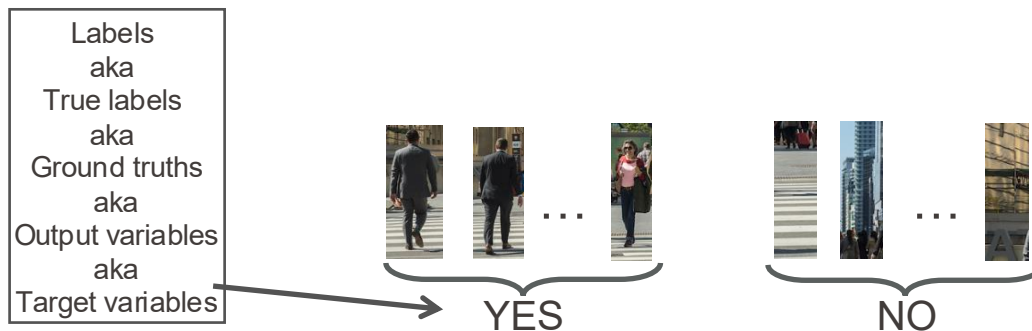
A computer program is said to:

- learn from experience **E**
- with respect to task **T**
- and performance measure **P**,

if its **P** on **T** improves with **E**

Machine learning to the rescue

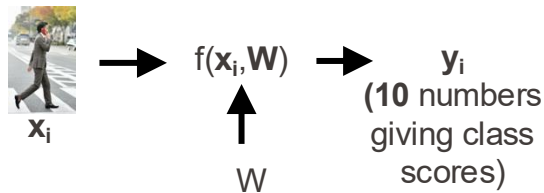
- Ex: Recognize Pedestrians from images:
 - **T**= Binary classification (yes or no)
 - **P**= Classification accuracy (% of correct predictions)
 - **E**=



- Input: x_i (Observed data, feature, representation...)
- Output: y_i (labels, desired outputs...)
- W : parameters of the ML model
- We want to learn the W for the mapping function $f : Y = f(X, W)$
- **Classification** when $f: \mathbb{R}^M \rightarrow \{1, \dots, C\}$
- **Regression** when $f: \mathbb{R}^M \rightarrow \mathbb{R}$
- ...
- f can be a linear function, or non-linear...
 - E.g., a linear function: $f(x_i, W) = w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots$
 - When the desired output is available, we want to:

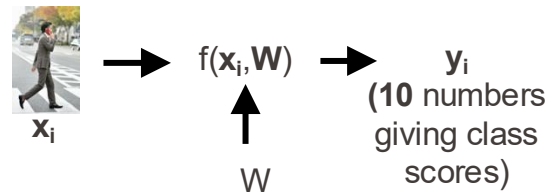
$$\min_W \quad \frac{1}{2} \sum_{i=1 \dots N} (f(x_i, W) - y_i)^2$$

=> optimization (e.g., gradient descent)



} Tasks T

- For classification, we often measure the accuracy of the model
- Accuracy is % correct outputs by the model
- We can also measure the error rate
- Given a dataset of examples (X_i, Y_i) , we split in 3 sets:
 - Training,
 - Validation,
 - Testing

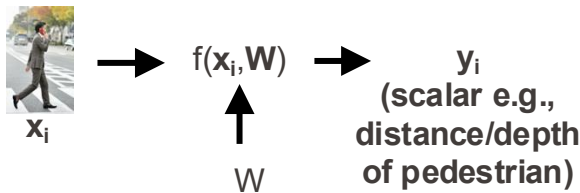


- Supervised learning
 - Labels are available (desired outputs)
- Semi-supervised learning
 - Some labels are missing
- Un-supervised learning
 - No labels
- Reinforcement learning
 - Labels (in form of rewards and punishments) are given only as feedback to the program's actions in a dynamic environment
- Active learning
 - limited set of labels are available and can be interactively selected
- Self-supervised learning
 - Use own signal as supervision



First ML example: Linear regression

- Input: x_i in $\mathbb{R}^M$ (observed vector, e.g., image of pedestrian)
- Output: y_i in $\mathbb{R}$ (target scalar, e.g., distance/depth of pedestrian)
- W : parameters of the model



- We want to learn the W for the mapping function $f : Y = f(X, W)$
- $f(x_i, W) = W^T X = w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots$

W can be seen as the weights that determine how each feature affects the prediction

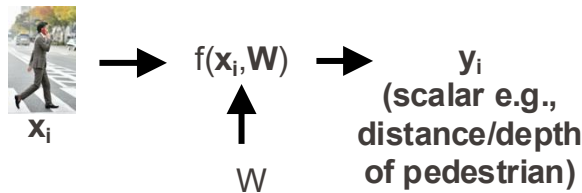
- We want to:
$$\min_W \frac{1}{2} \sum_{i=1}^N (f(x_i, W) - y_i)^2$$

Mean-Squared error (MSE)

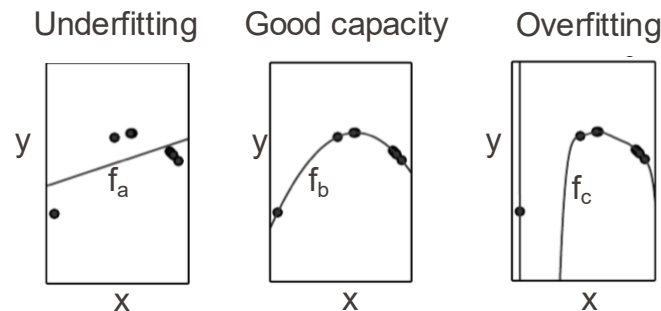
- $\nabla_w \text{MSE} = 0 \Rightarrow W = (X^T X)^{-1} X^T Y$

Note that for other functions such as logistic regression, there is no closed form and we use gradient descent

- Dataset of examples (X_i, Y_i) that we split in 3 sets: **Training, Validation, Testing**
- Use **optimization** techniques (e.g., stochastic gradient descent) to learn the parameters on the training set
- Compare different design choices using the validation set (learn **hyperparameters**)
- Evaluate final performance on test set (new images)
- The main challenge is **generalization**



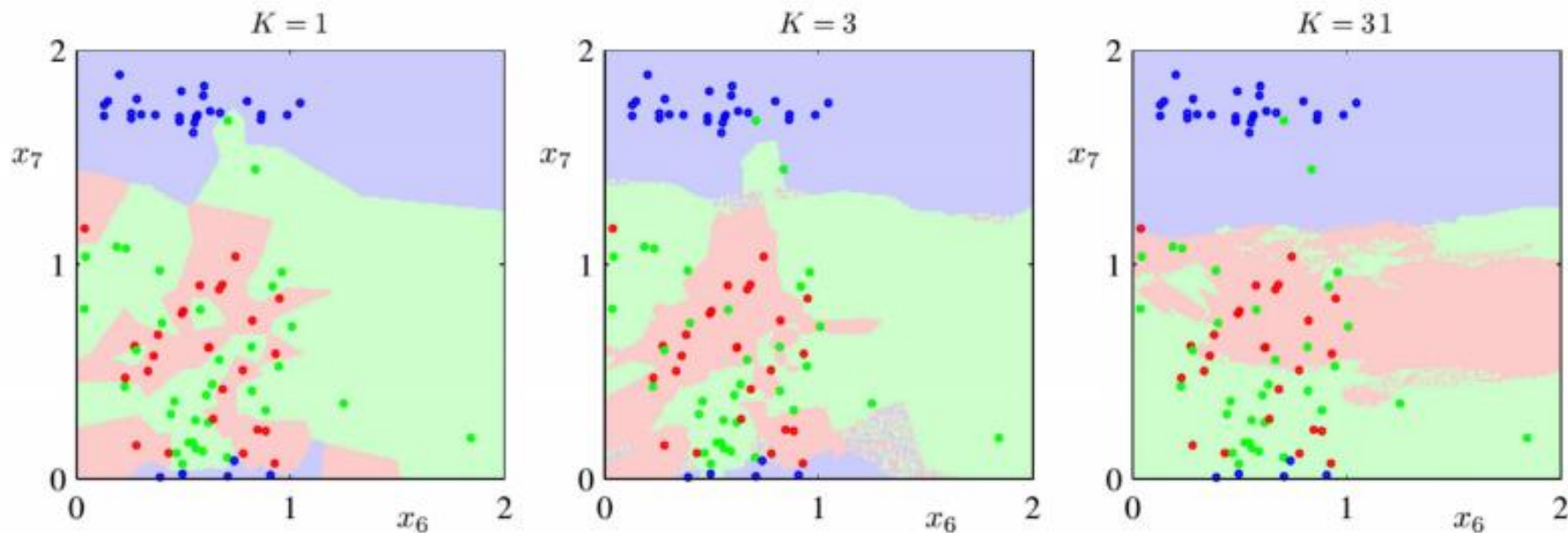
- Generalization is the ability to perform well on new unseen data (small Test error)
- In practice:
 - Training error $\leq$ Test error
- However,
 - when training error is not sufficiently low, the model **underfits**
 - when (Test error - Training error) is too big, the model **overfits**
- The solution is to control the **capacity**, e.g., choose the hypothesis space
 - E.g., in the linear regression case with input x and output y :
 - $f_a(x, W) = w_1 x$
 - $f_b(x, W) = w_1 x + w_2 x^2$
 - $f_c(x, W) = w_1 x + w_2 x^2 + \dots + w_5 x^5$



Non-Parametric approach: NEAREST NEIGHBOR

Hyperparameters

- What is the best value of k to use?
- What is the best **distance** to use?
- These are **hyperparameters**: choices about the model/algorithm that we set rather than learn



Example Dataset: CIFAR10

10 classes

50,000 training images

10,000 testing images

airplane

automobile

bird

cat

deer

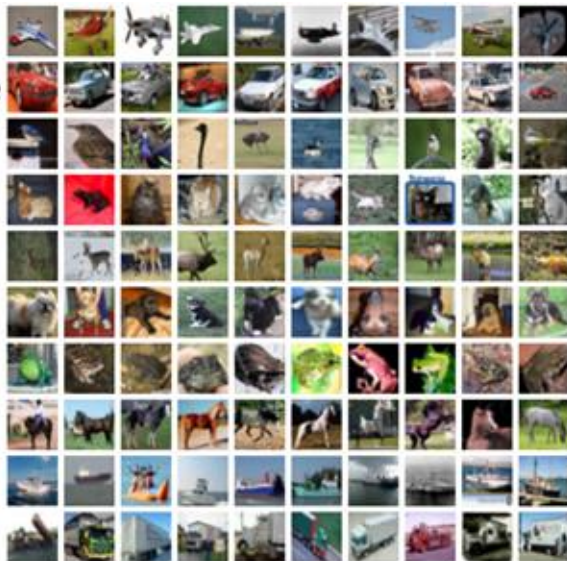
dog

frog

horse

ship

truck



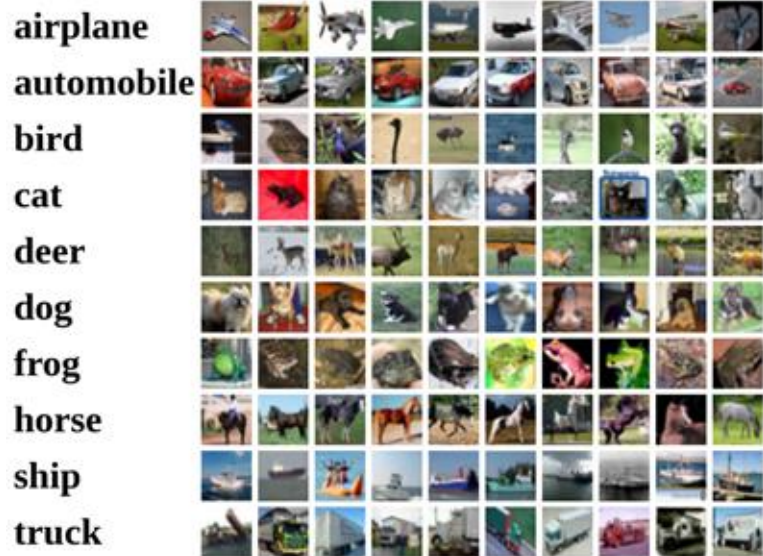
Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.

Example Dataset: CIFAR10

10 classes

50,000 training images

10,000 testing images



Test images and nearest neighbors



Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.


```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor classifier

Memorize training data

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor classifier

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

For each test image:
Find closest train image
Predict label of nearest image

Nearest Neighbor classifier

Q: With N examples, how fast are training and prediction?

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor classifier

Q: With N examples, how fast are training and prediction?

A: Train $O(1)$,
predict $O(N)$

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```


Nearest Neighbor classifier

Q: With N examples, how fast are training and prediction?

A: Train $O(1)$,
predict $O(N)$

This is bad: we want classifiers that are **fast** at prediction; **slow** for training is ok

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```


Hyperparameters

- What is the best value of **k** to use?
 - What is the best **distance** to use?
 - These are **hyperparameters**: choices about the model/algorithm that we set rather than learn
- ⇒ Very problem-dependent.
- ⇒ Must try them all out and see what works best.

Setting Hyperparameters

Your Dataset

Setting Hyperparameters

Your Dataset

Idea #1: Split data into **train** and **test**, choose hyperparameters that work best on test data

train

test

Setting Hyperparameters

Your Dataset

Idea #1: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data

train

test

Setting Hyperparameters

Your Dataset

Idea #1: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data

train

test

Idea #2: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

Better!

train

validation

test

Setting Hyperparameters

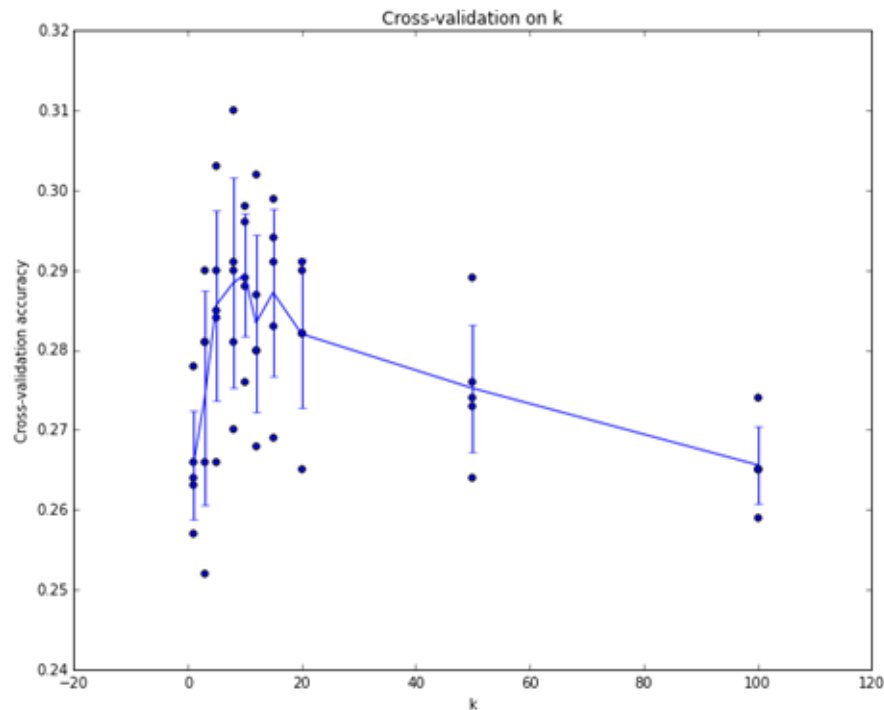
Your Dataset

Idea #3: Cross-Validation: Split data into **folds**,
try each fold as validation and average the results

fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test

Useful for small datasets, but not used too frequently in deep learning

Setting Hyperparameters



Example of
5-fold cross-validation
for the value of k .

Each point: single
outcome.

The line goes
through the mean, bars
indicated standard
deviation

(Seems that $k \approx 7$ works best
for this data)

So far, we learned...

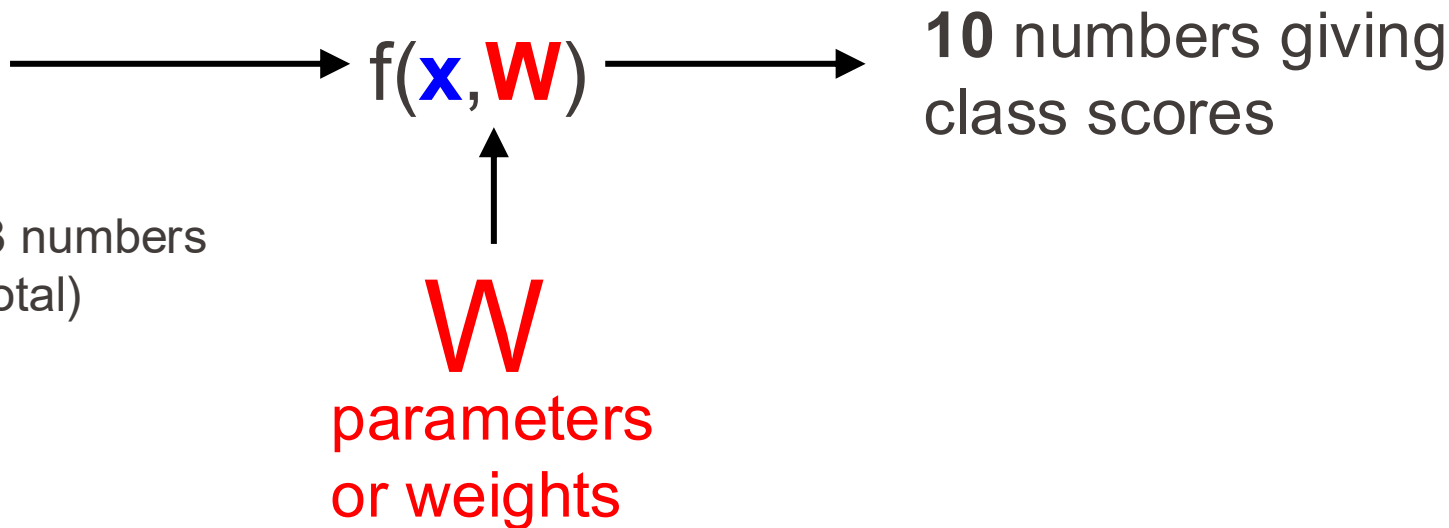
- In machine learning, we:
 - Start with a **training set** to learn **parameters** of the model
 - Change **hyperparameters** with **validation set**
 - Finally, evaluate our final model on **test set** (once)

- Nearest neighbor model is non-parametric approach
=> It does not have any parameters to learn (only hyperparameters)

Parametric approach: Linear classification

Parametric approach: Linear classification

Image



Parametric approach: Linear classification

$$f(x, W) = Wx$$

Image



10 numbers giving
class scores

Array of **32x32x3** numbers
(3072 numbers total)

W

parameters
or weights

Parametric approach: Linear classification

Image



Array of **32x32x3** numbers
(3072 numbers total)

$$f(x, W) = Wx$$

10x1 10x3072 3072x1

$$f(x, W)$$

10 numbers giving
class scores

W
parameters
or weights

Parametric approach: Linear classification

Image



Array of **32x32x3** numbers
(3072 numbers total)

$$f(x, W) = Wx + b$$

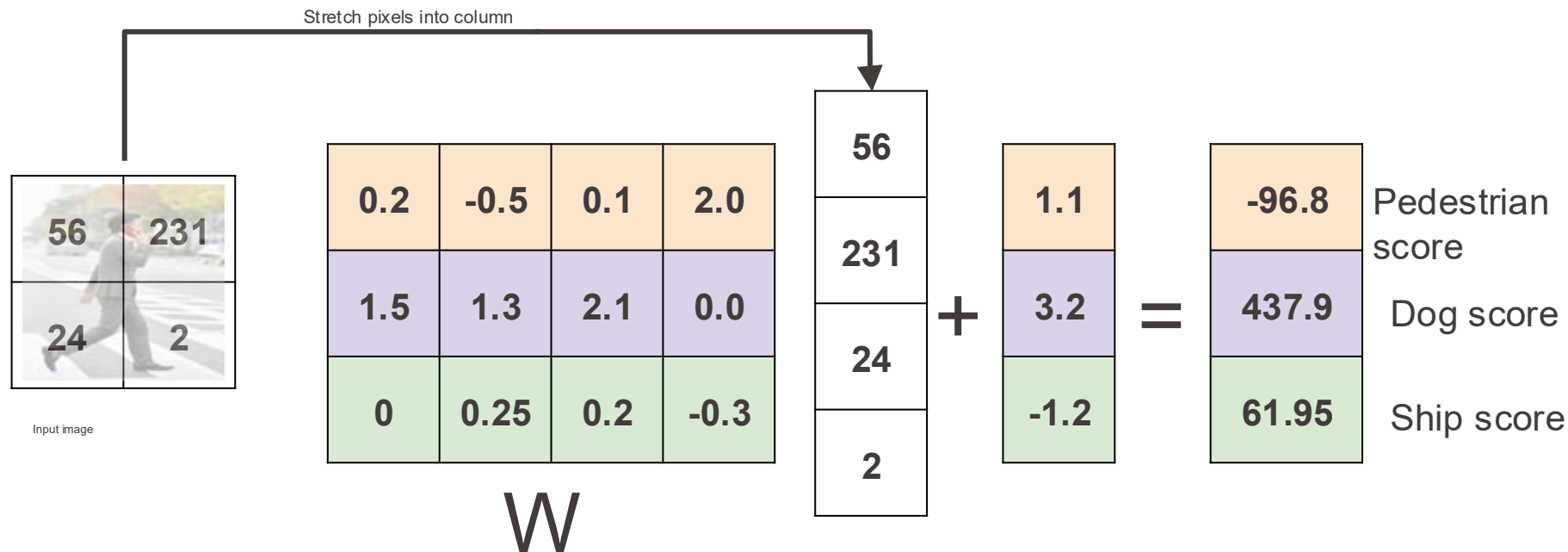
10×1 10×3072 3072×1 10×1

→ $f(x, W)$ → 10 numbers giving class scores

W

parameters
or weights

Example with an image with 4 pixels, and 3 classes (Pedestrian/dog/ship)



With 3 images...

To define a (linear) score function:

$$f(x, W) = Wx + b$$



Example class
scores for 3 images
for some W :

Pedestrian	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

How can we tell
whether this W is
good or bad?



- Suppose: 3 training examples, 3 classes.
- With some W the scores are $s = f(x, W) = Wx$
- A loss function tells how good our current classifier is
- Given a dataset of examples $\{(x_i, y_i)\}_{i=1}^N$

Where x_i is image and
 y_i is (integer) label

- Softmax classifier (Multinomial Logistic Regression)

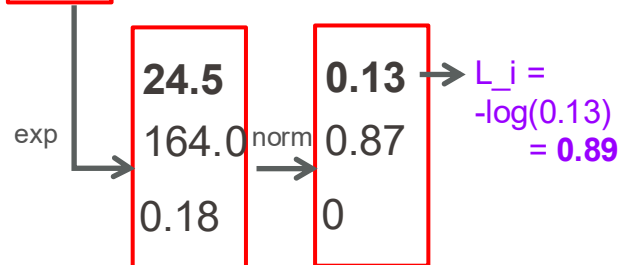
$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

- Max log likelihood, aka Min negative log likelihood

$$L_i = -\log P(Y = y_i | X = x_i) = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$



Ped	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1



Q: What is the min/max possible loss L_i ?

- Suppose: 3 training examples, 3 classes.
- With some W the scores are $s = f(x, W) = Wx$
- A loss function tells how good our current classifier is
- Given a dataset of examples $\{(x_i, y_i)\}_{i=1}^N$

Where x_i is image and
 y_i is (integer) label

- Softmax classifier (Multinomial Logistic Regression)

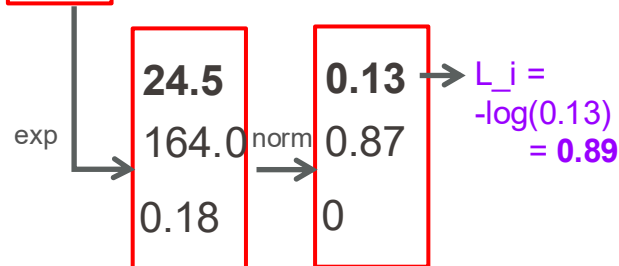
$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

- Max log likelihood, aka Min negative log likelihood

$$L_i = -\log P(Y = y_i | X = x_i) = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$



Ped	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1



Q: Usually at initialization W is small so all $s \approx 0$. What is the loss?

- Suppose: 3 training examples, 3 classes.
- With some W the scores are $s = f(x, W) = Wx$
- A loss function tells how good our current classifier is
- Given a dataset of examples $\{(x_i, y_i)\}_{i=1}^N$

Where x_i is image and
 y_i is (integer) label

- Softmax classifier (Multinomial Logistic Regression)

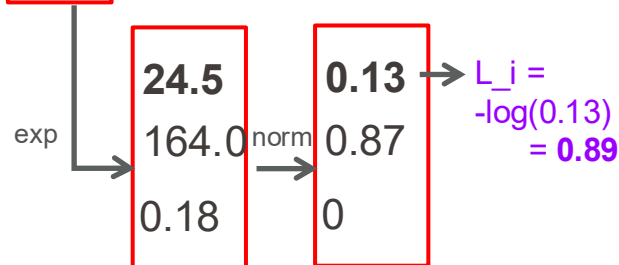
$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

- Max log likelihood, aka Min negative log likelihood

$$L_i = -\log P(Y = y_i | X = x_i) = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$



Ped	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1



- Suppose: 3 training examples, 3 classes.
- With some W the scores are $s = f(x, W) = Wx$
- A loss function tells how good our current classifier is
- Given a dataset of examples $\{(x_i, y_i)\}_{i=1}^N$

Where x_i is image and
 y_i is (integer) label



Ped	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

- Support Vector Machine loss:

$$\begin{aligned}
 L_i &= \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_j - s_{y_i} + 1 & \text{otherwise} \end{cases} \\
 &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)
 \end{aligned}$$

- Suppose: 3 training examples, 3 classes.
- With some W the scores are $s = f(x, W) = Wx$
- A loss function tells how good our current classifier is
- Given a dataset of examples $\{(x_i, y_i)\}_{i=1}^N$

Where x_i is image and
 y_i is (integer) label

- Support Vector Machine loss:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$



Ped	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9		

$$\begin{aligned}
 &= \max(0, 5.1 - 3.2 + 1) \\
 &\quad + \max(0, -1.7 - 3.2 + 1) \\
 &= \max(0, 2.9) + \max(0, -3.9) \\
 &= 2.9 + 0 \\
 &= 2.9
 \end{aligned}$$

- Suppose: 3 training examples, 3 classes.
- With some W the scores are $s = f(x, W) = Wx$
- A loss function tells how good our current classifier is
- Given a dataset of examples $\{(x_i, y_i)\}_{i=1}^N$

Where x_i is image and
 y_i is (integer) label

- Support Vector Machine loss:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$



Ped	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	

$$\begin{aligned}
 &= \max(0, 1.3 - 4.9 + 1) \\
 &\quad + \max(0, 2.0 - 4.9 + 1) \\
 &= \max(0, -2.6) + \max(0, -1.9) \\
 &= 0 + 0 \\
 &= 0
 \end{aligned}$$

- Suppose: 3 training examples, 3 classes.
- With some W the scores are $s = f(x, W) = Wx$
- A loss function tells how good our current classifier is
- Given a dataset of examples $\{(x_i, y_i)\}_{i=1}^N$

Where x_i is image and
 y_i is (integer) label



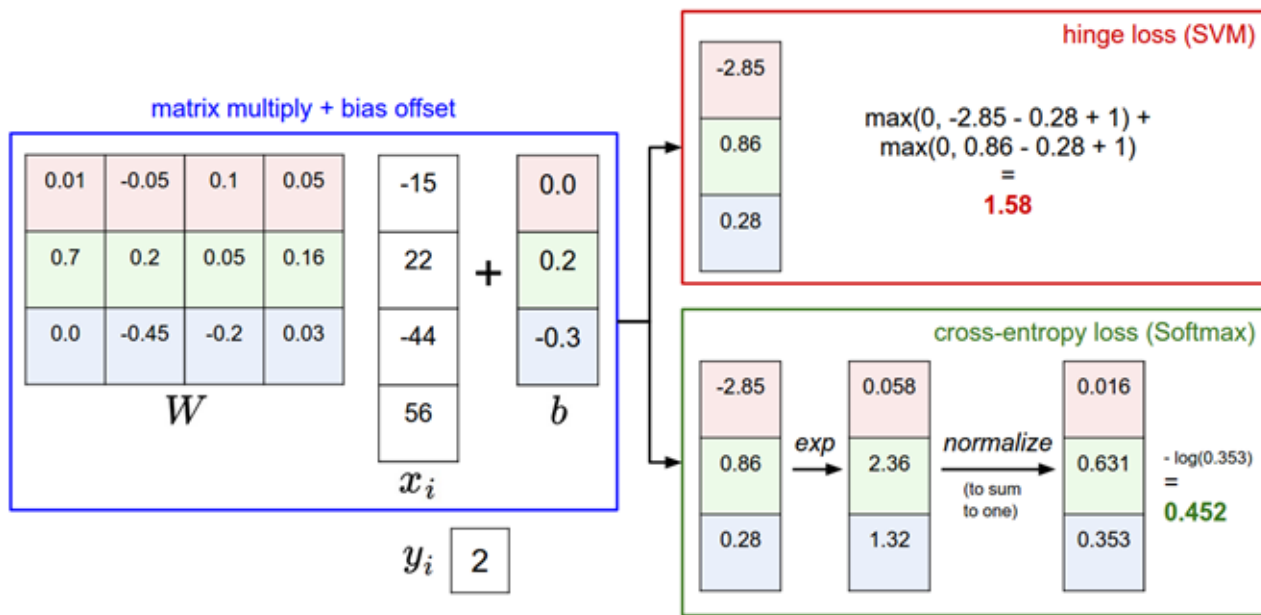
Ped	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$\begin{aligned}
 &= \max(0, 2.2 - (-3.1) + 1) \\
 &\quad + \max(0, 2.5 - (-3.1) + 1) \\
 &= \max(0, 6.3) + \max(0, 6.6) \\
 &= 6.3 + 6.6 \\
 &= 12.9
 \end{aligned}$$

Hinge Loss VS Cross-Entropy Loss

- We saw how to quantify good W



How to find the best parameters W ?

- Optimization
 - Techniques to find the best parameters **W**



- refers to the task of either minimizing or maximizing some function $L(w)$ by altering w .
- $L(w)$ is often referred to as objective function/criterion or cost/error/loss function in case of minimization:

$$L = \frac{1}{N} \sum_i^N L_i(f(x_i, W), y_i)$$

- Where in the case of a Hinge loss:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

- And a linear classifier:

$$s = f(x, W) = Wx$$

where N is the # samples)



Strategy #1: A first very bad solution: random search

```
# assume X_train is the data where each column is an example (e.g. 3073 x 50,000)
# assume Y_train are the labels (e.g. 1D array of 50,000)
# assume the function L evaluates the loss function

bestloss = float("inf") # Python assigns the highest possible float value
for num in xrange(1000):
    W = np.random.randn(10, 3073) * 0.0001 # generate random parameters
    loss = L(X_train, Y_train, W) # get the loss over the entire training set
    if loss < bestloss: # keep track of the best solution
        bestloss = loss
        bestW = W
    print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)

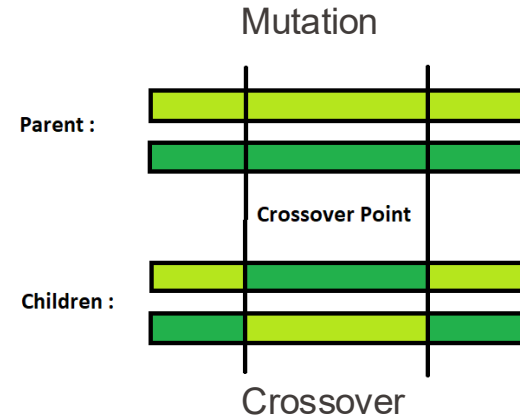
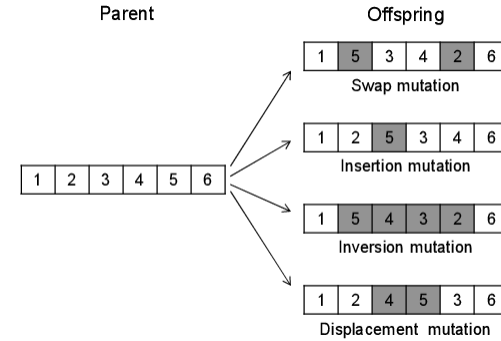
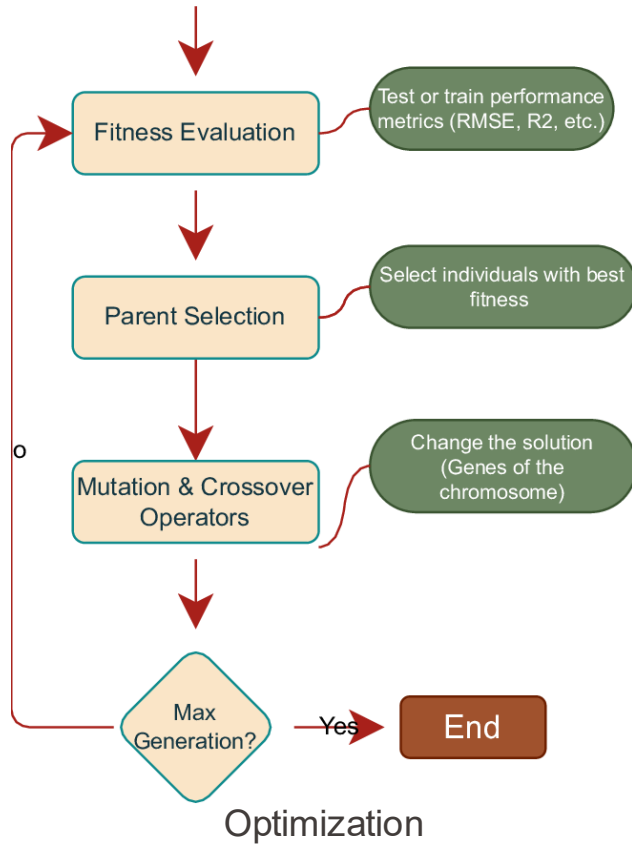
# prints:
# in attempt 0 the loss was 9.401632, best 9.401632
# in attempt 1 the loss was 8.959668, best 8.959668
# in attempt 2 the loss was 9.044034, best 8.959668
# in attempt 3 the loss was 9.278948, best 8.959668
# in attempt 4 the loss was 8.857370, best 8.857370
# in attempt 5 the loss was 8.943151, best 8.857370
# in attempt 6 the loss was 8.605604, best 8.605604
# ... (truncated: continues for 1000 lines)
```

Lets see how well this works on the test set...

```
# Assume X_test is [3073 x 10000], Y_test [10000 x 1]
scores = Wbest.dot(Xte_cols) # 10 x 10000, the class scores for all test examples
# find the index with max score in each column (the predicted class)
Yte_predict = np.argmax(scores, axis = 0)
# and calculate accuracy (fraction of predictions that are correct)
np.mean(Yte_predict == Yte)
# returns 0.1555
```

15.5% accuracy! not bad!
(SOTA is ~95%)

Strategy #2: Genetic Algorithms



Very slow; not as good but better than random

Strategy #3: Follow the slope

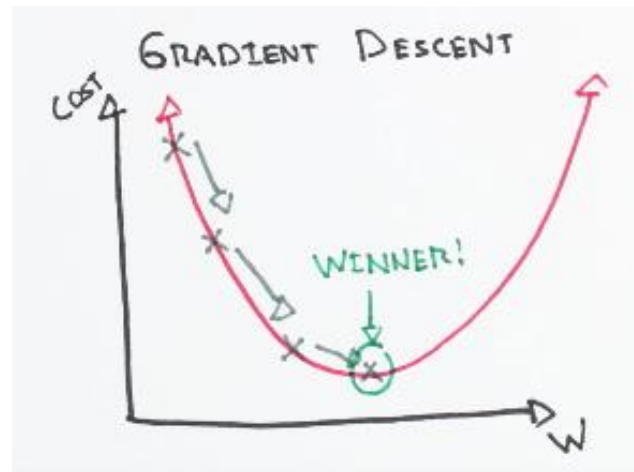


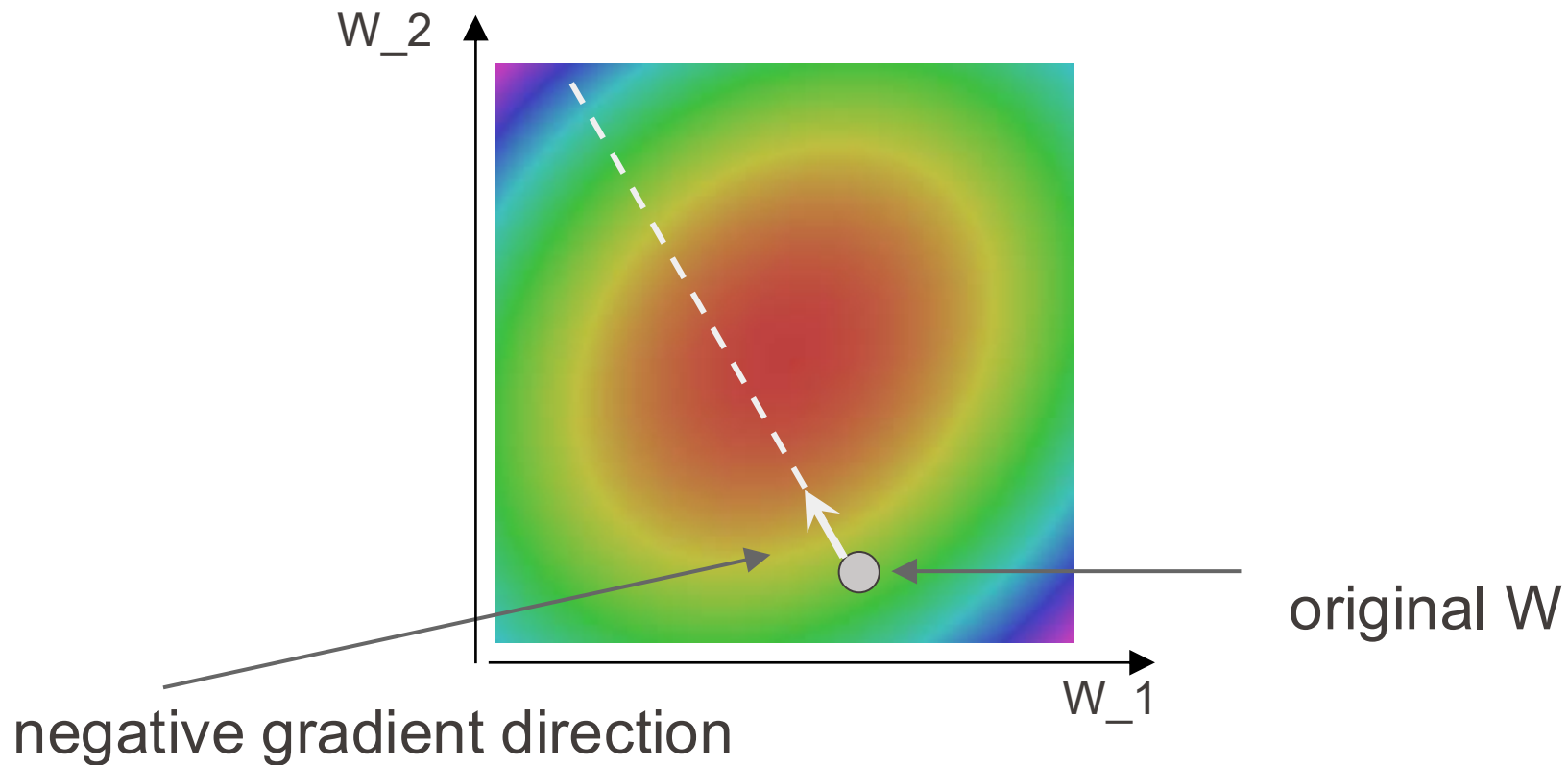
Strategy #3: Follow the slope

- In 1-dimension, the derivative of a function:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- In multiple dimensions, the **gradient** is the vector of (partial derivatives) along each dimension
- The direction of steepest descent is the **negative gradient**






```
# Vanilla Gradient Descent
```

```
while True:
```

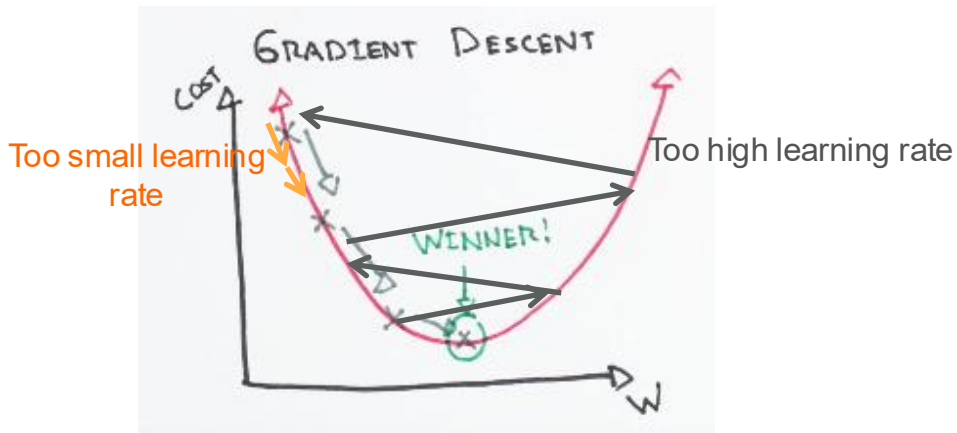
```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```



Learning rate

Importance of Learning Rates



⇒ Annealing **learning rate**: step decay, exp decay, $1/t$ decay...

≤ hyperparameter

Stochastic Gradient Descent (SGD)

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W)$$

Full sum expensive
when N is large!

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W)$$

Approximate sum
using a **minibatch** of
examples
32 / 64 / 128 common

```
# Vanilla Minibatch Gradient Descent
```

```
while True:
```

```
    data_batch = sample_training_data(data, 256) # sample 256 examples
```

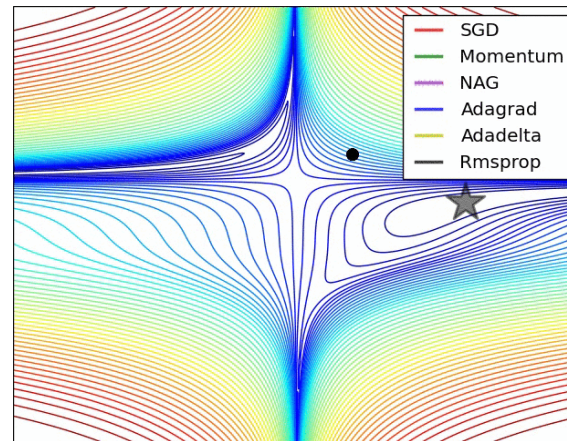
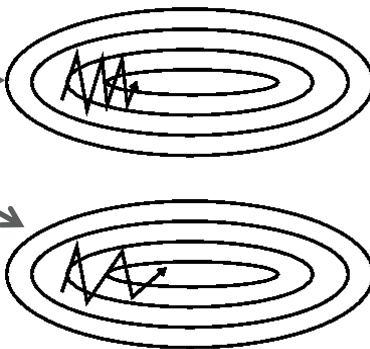
```
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```

Popular Gradient descent optimization methods

1st order methods

- SGD
- Momentum update
- Nesterov Momentum
- Adagrad
- RMSprop
- Adam
- ...



2nd order methods

- Newton (based on the Hessian matrix H , the 2nd order partial derivatives)
- Quasi Newton methods (approx Hessian) e.g., L-BFGS...

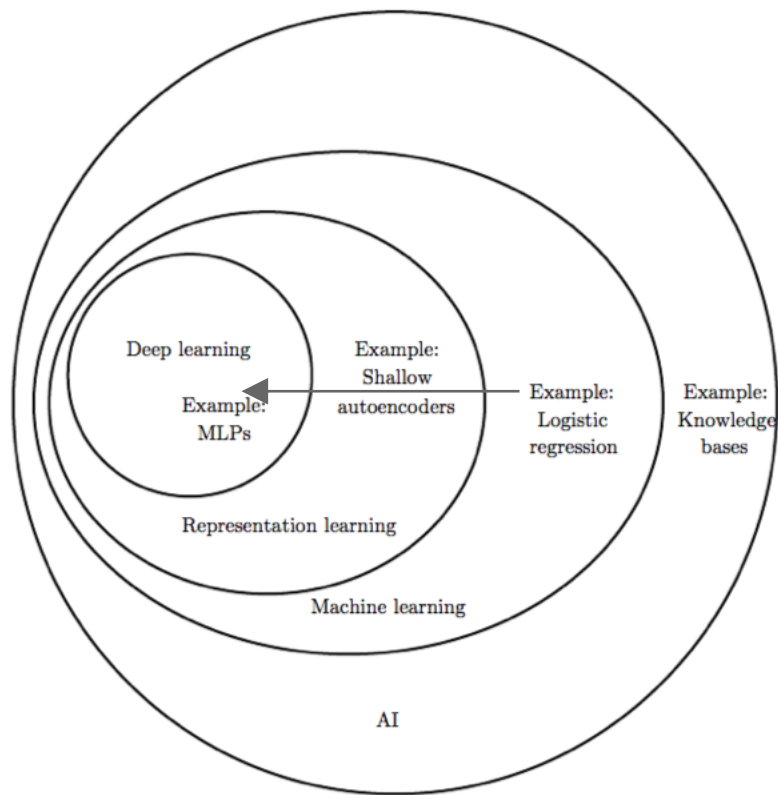
$$x \leftarrow x - [Hf(x)]^{-1} \nabla f(x)$$

- Dataset of (x,y)
- Score function, e.g., linear classifier
 $f(x,W) = Wx+b$
- Loss functions e.g., softmax loss
(quantifies how good is W)
- Optimization methods (find the best W)

We will see:

- **New functional for the score function**
 $f(x,W)$ e.g., Neural Networks

Where are we going



- Representation learning

- Book: Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville
(<http://www.deeplearningbook.org/>)

- Class on CNN: <http://cs231n.stanford.edu/>

- A few useful things to know about ML by Pedro Domingos

<https://homes.cs.washington.edu/~pedrod/papers/cacm12.pdf>

- Python Numpy tutorial
<http://cs231n.github.io/python-numpy-tutorial/>

- K Nearest Neighbors:
<http://vision.stanford.edu/teaching/cs231n-demos/knn/>
- Linear classify demo:
<http://vision.stanford.edu/teaching/cs231n-demos/linear-classify/>
- A visual introduction to machine learning:
<http://www.r2d3.us/visual-intro-to-machine-learning-part-1/>
- Nice interactive visualization on some Machine Learning techniques:
<https://distill.pub/>